

Bài toán CV lõi: nhận dạng, phát hiện, phân vùng (Core Vision Tasks: Recognition, Detection, Segmentation)

PHỤ THUỘC C/13-kien-truc-backbone (backbone, stride, attention); B/09-thiet-ke-ham-mat-mat (loss đa nhiệm, mất cân bằng); B/07-chia-du-lieu-va-danh-gia (mAP, mIoU, đánh giá theo lát). [T3]

TÓM TẮT

Chương này lắp các “đầu” bài toán lên trên backbone của chương trước, và đọc toàn bộ lịch sử detection lẫn segmentation theo một trục duy nhất: từng bước nuốt các heuristic bên ngoài vào bên trong mạng, tức biến một khối không nhận gradient thành một khối được tối ưu cho đúng mục tiêu cuối. Đọc xong bạn phát biểu được một bài toán thì giác mới dưới dạng đúng, chọn được họ mô hình từ ràng buộc chứ không từ bảng xếp hạng, và đọc được một bảng so sánh detector với đúng mức hoài nghi. Nếu chỉ nhớ một điều: những chi tiết không được viết trong abstract — cách gán mẫu, tỉ lệ mẫu dương/âm, lịch huấn luyện — thường giải thích chênh lệch kết quả nhiều hơn hình vẽ kiến trúc ở trang đầu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và huấn luyện** — mô hình là một hàm có hàng triệu tham số biến ảnh thành dự đoán; huấn luyện là chỉnh dần các tham số đó để dự đoán khớp nhãn, bằng cách đi ngược hướng *gradient* của hàm mất mát.
- **hàm mất mát (loss function)** — một số duy nhất đo mức sai của dự đoán. Đây là thứ định nghĩa “đúng” nghĩa là gì, nên đối hàm mất mát là đối bài toán chứ không phải đối mô hình.
- **backbone, neck, head** — backbone biến ảnh thành lưới đặc trưng; neck trộn các lưới ở nhiều độ phân giải; head là phần nhỏ phía trên sinh ra đầu ra của một bài toán cụ thể. Ba bài toán khác nhau thường dùng chung backbone và chỉ khác head.
- **chân trị (ground truth)** — đáp án do người gán cho mỗi ảnh — khung, mask, hay nhãn lớp. Mô hình chỉ học được đúng những gì nhãn nói, kể cả khi nhãn sai.
- **IoU** — diện tích giao chia diện tích hợp của hai vùng; bằng 1 khi trùng khít, 0 khi rời nhau. Nó là đơn vị đo cơ bản của cả detection lẫn segmentation.
- **mAP và mIoU** — hai thước đo tổng: mAP cho detection (diện tích dưới đường precision-recall, trung bình theo lớp và theo ngưỡng IoU), mIoU cho segmentation (IoU trung bình theo lớp).
- **heuristic** — một quy tắc viết tay, không có tham số học được, do đó không nhận gradient và không tự chỉnh theo lỗi cuối. Ví dụ: NMS, ngưỡng IoU khi gán mẫu, công thức chọn tầng của FPN.

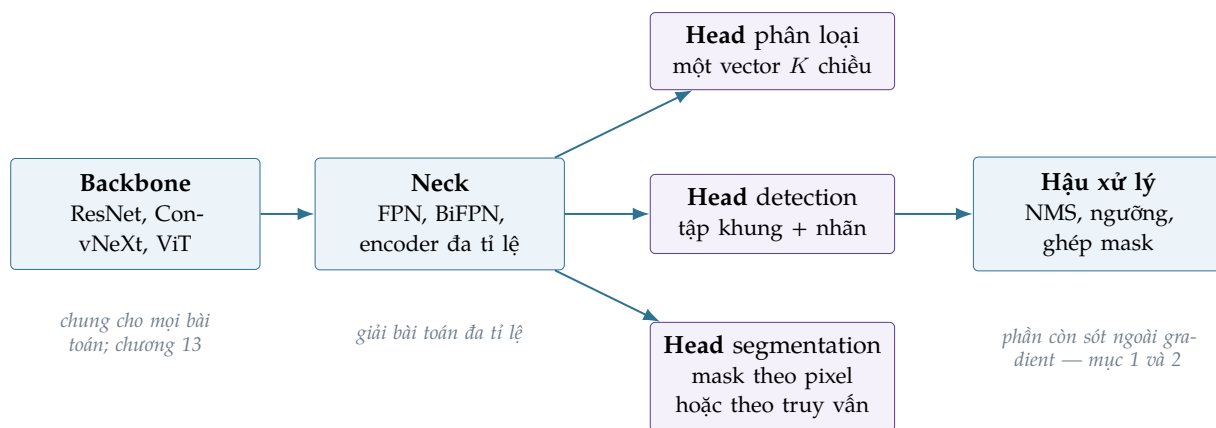
CÂU HỎI MỞ ĐẦU

- Q1. Vì sao thay đúng một quy tắc gán mẫu — một hàm chục dòng nằm trong code, không có trong abstract — lại đổi kết quả nhiều hơn thay cả backbone?
- Q2. R-CNN chạy backbone hai nghìn lần mỗi ảnh, Fast R-CNN chạy một lần. Vì sao bản một lần lại *chính xác hơn* chứ không chỉ nhanh hơn?
- Q3. NMS chỉ là một vòng lặp xóa khung trùng. Vì sao suốt mười năm không ai “nuốt” được nó vào mạng, trong khi selective search thì nuốt được ngay?

- Q4.** Nếu bỏ đúng một phép làm tròn toạ độ mà chất lượng mask nhảy vọt, điều đó nói gì về việc bạn nên tin phần nào của một paper?
- Q5.** Semantic, instance và panoptic segmentation từng là ba code base riêng. Việc chúng gộp được vào một mô hình là dấu hiệu của cái gì?

Chương trước cho bạn cái máy trích đặc trưng; chương này hỏi phải gắn cái gì lên trên nó để trả lời những câu hỏi mà thế giới thật đặt ra. Ba câu hỏi kinh điển — *ảnh này là gì, có những vật gì ở đâu, mỗi pixel thuộc về cái gì* — trông như ba mức độ chi tiết tăng dần của cùng một bài toán. Nhưng chỉ câu thứ nhất có đầu ra kích thước cố định, và chính điều đó chia đôi chương này: mọi khó khăn kỹ thuật của detection và segmentation đều bắt nguồn từ việc đầu ra là một *tập hợp* có lực lượng thay đổi, thứ mà bộ máy gradient không biết chấm điểm trực tiếp.

Lịch sử của lĩnh vực này, đọc theo đúng trục, là một câu chuyện duy nhất: **từng bước nuốt các heuristic bên ngoài vào bên trong mạng**. Selective search bị RPN nuốt, thiết kế đa tỉ lệ thủ công bị FPN nuốt, lấy mẫu cứng bị focal loss nuốt, NMS bị Hungarian matching nuốt. Mỗi lần nuốt, một khối vốn không nhận được gradient trở thành một khối được tối ưu cho đúng mục tiêu cuối, và gần như lần nào hệ thống cũng vừa nhanh hơn vừa chính xác hơn. Đọc theo mạch đó, bạn không chỉ nhớ được lịch sử mà còn đoán được bước tiếp theo của bất kỳ dòng mô hình nào: hãy liệt kê những gì vẫn còn nằm ngoài.



Hình 1.1. Giải phẫu chung của một hệ thống thị giác hiện đại. Ba đầu bài toán khác nhau dùng chung backbone và neck; cái phân biệt chúng là *dạng đầu ra* và do đó là hàm mất mát. Khối hậu xử lý bên phải là phần duy nhất không nhận gradient, và toàn bộ chương này là câu chuyện thu nhỏ khối đó lại.

Hình 1.1 là bản đồ nên giữ trong đầu suốt chương: mỗi khi gặp một mô hình mới, hãy hỏi nó thay đổi khối nào trong bốn khối này.

Chương chia làm bốn mục theo đúng thứ tự phụ thuộc đó. Mục 1 đi hết dòng hai giai đoạn, nơi mẫu hình “nuốt heuristic” hiện ra rõ nhất và nơi các khái niệm nền — anchor, RoI, NMS — được định nghĩa cho cả chương. Mục 2 bỏ giai đoạn đề xuất để lấy tốc độ, và đúng lúc đó một vấn đề mới lộ ra mà mục 1 chưa hề gặp: mất cân bằng foreground–background, thứ được giải quyết ở tầng hàm mất mát chứ không ở tầng kiến trúc. Mục 2 cũng là nơi đặt bài học trung tâm của cả chương: cách gán mẫu cho các vị trí dự đoán — thành phần hiểm khi nằm trong abstract và thường nằm trong code — giải thích phần lớn chênh lệch giữa hai detector, nhiều hơn cả kiến trúc. Mục 3 chuyển sang segmentation. Ở đó cùng một mâu thuẫn “ngữ nghĩa sâu chống lại định vị chính xác” được tấn công từ bốn phía. Mục kết thúc bằng một lần đổi phát biểu, hợp nhất semantic, instance và panoptic. Lần đổi phát biểu đó mượn nguyên ý tưởng dự đoán tập hợp của mục 2, nên hai mục phải đọc theo thứ tự. Mục 4 gom các bài toán lân cận mà kỹ sư gặp hằng ngày: pose, OCR, retrieval, cùng hai chế độ phân bố dữ liệu mà mọi con số tổng đều che giấu.

Nếu bạn chỉ có thời gian cho một phần, hãy đọc mục 2. Nó chứa cái mà roadmap gọi là bài học lặp lại của cả lĩnh vực, và nó thay đổi cách bạn đọc mọi bảng kết quả detection về sau.

Bản đồ chương 14 — Bài toán CV lõi (Core Vision Tasks)

TÓM TẮT

Chương 14 gồm bốn mục, chia theo *dạng đầu ra* chứ không theo thứ tự lịch sử: hai mục đầu về đầu ra dạng tập khung (detection), mục ba về đầu ra dạng mask, mục bốn về những dạng đầu ra còn lại mà kỹ sư gặp hằng ngày. Đọc bản đồ này để biết mục nào là cốt lõi phải nắm chắc, mục nào tra khi cần, và thứ tự phụ thuộc giữa chúng. Nếu chỉ nhớ một điều: mục 2 là mục cốt lõi của cả chương, vì nó chứa bài học “cách gán mẫu quyết định nhiều hơn kiến trúc”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mục (section) và chương** — chương là một chủ đề lớn; mỗi mục là một file riêng đọc độc lập được. Số thứ tự trong tên thư mục phản ánh *thứ tự nên đọc*, không phải thứ tự lịch sử.
- **cốt lõi và tham khảo** — “cốt lõi” là phần phải nắm chắc và quay lại nhiều lần; “tham khảo” là phần biết-là-có, đọc kỹ khi thật sự gặp bài toán đó.
- **detection và segmentation** — detection trả về danh sách khung chữ nhật kèm nhãn; segmentation trả về nhãn cho từng pixel. Khác biệt cốt lõi là *dạng đầu ra*, và nó kéo theo hàm mất mát lẫn thước đo.
- **gán mẫu (label assignment)** — quy tắc quyết định vị trí dự đoán nào phải học vật nào. Nó nằm trong định nghĩa của hàm mất mát chứ không phải trong kiến trúc — lý do nó là bài học trung tâm của chương.

Có gì trong chương này

- **Mục 1 — Detection hai giai đoạn (cốt lõi)**. R-CNN → Fast R-CNN → Faster R-CNN với RPN → FPN → Mask R-CNN với RoIAlign. Đây là nơi định nghĩa các khái niệm nền dùng lại suốt chương: anchor, RoI, NMS, IoU, loss đa nhiệm.
- **Mục 2 — Một giai đoạn và cuộc đánh đổi tốc độ (cốt lõi, đọc kỹ nhất)**. YOLO/SSD, mất cân bằng nền–vật thể, focal loss, anchor-free, và thực sự quan trọng: **gán mẫu (label assignment)** từ IoU cố định qua ATSS tới SimOTA, rồi DETR bỏ NMS bằng Hungarian matching.
- **Mục 3 — Phân vùng ảnh và sự hợp nhất (cốt lõi)**. FCN, U-Net, DeepLab với atrous/ASPP, SegFormer, Mask2Former, rồi bước dịch sang mô hình nền tảng nhận prompt.
- **Mục 4 — Các bài toán lân cận (tham khảo có trọng điểm)**. Pose, OCR, metric learning và truy hồi, fine-grained và long-tail. Đọc lướt để biết là có, quay lại khi thật sự gặp.

Vì sao chia như vậy

Ba câu hỏi kinh điển của thị giác — *ảnh này là gì, có gì ở đâu, mỗi pixel thuộc về cái gì* — trông như ba mức chi tiết của cùng một bài toán, nhưng chỉ câu đầu có đầu ra kích thước cố định. Chương chia theo đúng ranh giới đó: mục 1 và 2 xử lý đầu ra dạng tập hợp khung, mục 3 xử lý đầu ra dạng mask, mục 4 gom các dạng còn lại. Cách chia này quan trọng vì *dạng đầu ra* kéo theo hàm mất mát, metric và chế độ hồng — ba thứ khó đổi hơn kiến trúc rất nhiều.

Thứ tự đọc và phụ thuộc

1. Đọc C/13-kien-truc-backbone trước; toàn chương giả định bạn hiểu stride, trường tiếp nhận và feature map đa tỉ lệ.
2. Mục 1 trước mục 2: mất cân bằng nền-vật thể chỉ lộ ra *sau khi* bỏ giai đoạn đề xuất, nên đọc ngược thứ tự thì mất luôn nguyên nhân.
3. Mục 2 trước mục 3: khung truy vấn của Mask2Former mượn nguyên ý tưởng dự đoán tập hợp của DETR.
4. Mục 4 độc lập, đọc lúc nào cũng được.
5. Đọc B/09-thiet-ke-ham-mat-mat song song với mục 2 nếu phần focal loss chưa rõ, và B/07-chia-du-lieu-va-danh-gia song song với mục 4.

Nếu chỉ có một buổi

Đọc phần đặt vấn đề của mục 1, rồi đọc trọn mục 2. Mục 2 chứa bài học lặp lại của cả lĩnh vực — rằng những chi tiết không được viết trong abstract thường quyết định kết quả nhiều hơn kiến trúc — và bài học đó thay đổi cách bạn đọc mọi bảng kết quả về sau.

1.1 Detection hai giai đoạn nuốt dần heuristic (Two-Stage Detection)

PHỤ THUỘC C/13-kien-truc-backbone/01-dong-cnn (feature map, stride, trường tiếp nhận (*receptive field*)); B/09-thiet-ke-ham-mat-mat (loss đa nhiệm); B/05-thiet-ke-bai-toan (bộ ba đầu vào – đầu ra – tín hiệu giám sát).

TÓM TẮT

Mục này theo dấu bốn thể hệ detection hai giai đoạn: R-CNN, Fast R-CNN, Faster R-CNN với FPN, và Mask R-CNN. Chúng không phải bốn ý tưởng rời rạc, mà là bốn lần thay một bước thủ công bằng một khối học được. Đọc xong, cầm bất kỳ paper detection nào bạn cũng chỉ ra được hai thứ: khối nào nằm trong tầm với của gradient, và khối nào là heuristic cố định. Từ đó bạn đoán được bước cải tiến hợp lý tiếp theo. Đó là kỹ năng dự đoán, không phải ghi nhớ. Nếu chỉ nhớ một điều: ranh giới “cái gì nhận được gradient” dịch xuống qua từng thể hệ. Hai thứ sống sót được là triết phi cực đại và cách gán mẫu. Chúng sống sót vì lý do cấu trúc, không phải ngẫu nhiên, nên gỡ chúng đòi hỏi đổi phát biểu bài toán chứ không đổi kiến trúc.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, ánh xạ ảnh đầu vào thành dự đoán; “huấn luyện” là tìm bộ tham số làm dự đoán khớp nhãn nhất có thể.
- **gradient** — vector đạo hàm của hàm mất mát theo từng tham số; nó chỉ hướng phải chỉnh tham số để lỗi giảm. Một khối “không nhận được gradient” là khối không tự chỉnh được, dù nó gây ra lỗi.
- **backbone** — phần mạng dùng chung để biến ảnh thành *feature map* — một lưới vector đặc trưng, mỗi ô mô tả nội dung của một vùng ảnh. Phần gần phía trên để trả lời một bài toán cụ thể gọi là *head*.
- **stride** — tỉ lệ thu nhỏ giữa ảnh và feature map. Stride 16 nghĩa là một ô đặc trưng ứng với một vùng 16×16 pixel ảnh, nên mọi toạ độ tính trên feature map đều thô tới 16 pixel.
- **IoU** — diện tích giao chia diện tích hợp của hai khung chữ nhật; bằng 1 khi trùng khít, 0 khi rời nhau. Đây là thước đo “khung dự đoán có đúng chỗ không”.

- **anchor** — một khung chữ nhật mẫu, kích thước và tỉ số cạnh định sẵn, rải đều trên mọi vị trí của feature map. Mô hình không đoán khung từ số không mà đoán *độ lệch* so với anchor gần nhất — một bài toán dễ hơn nhiều.
- **proposal (đề xuất vùng)** — một vùng ảnh ứng viên “có thể chứa vật gì đó”, sinh ra ở giai đoạn một để giai đoạn hai chỉ phải phân loại vài trăm vùng thay vì hàng chục nghìn.
- **NMS** — bước hậu xử lý xoá các khung trùng lặp: giữ khung điểm cao nhất, xoá mọi khung có IoU với nó vượt ngưỡng, lặp lại. Nó là thuật toán viết tay, không học được.
- **mAP** — thước đo chuẩn của detection: với mỗi lớp, vẽ đường precision-recall rồi lấy diện tích dưới đường đó, sau đó trung bình trên mọi lớp và trên nhiều ngưỡng IoU.

Đặt vấn đề

Phân loại ảnh có một đặc tính dễ chịu mà ta ít khi để ý: đầu ra của nó có kích thước cố định. Một ảnh vào, một vector K chiều ra, và toàn bộ bộ máy huấn luyện bằng *gradient* chạy trơn tru vì **hàm mất mát** (*loss function*) được định nghĩa trên một đối tượng có hình dạng biết trước. **Phát hiện vật thể** (*object detection*) phá vỡ đúng đặc tính đó. Số vật thể trong ảnh thay đổi từ không tới hàng trăm, mỗi vật cần một nhãn và một vị trí, nên đầu ra là một **tập hợp** (*set*) có lực lượng thay đổi. Không có cách hiển nhiên nào viết một hàm mất mát khả vi trên tập hợp, và đây chính là nút thắt mà toàn bộ lịch sử detection xoay quanh.

Cách né đầu tiên, và trong nhiều năm là cách duy nhất, là biến bài toán tập hợp thành nhiều bài toán phân loại: đề xuất trước một tập ứng viên vùng ảnh, rồi phân loại từng ứng viên độc lập. Mẹo này giải quyết được vấn đề hình dạng đầu ra, nhưng nó đẩy độ khó sang chỗ khác — sang câu hỏi *ai sinh ra các ứng viên đó*. Trong R-CNN, câu trả lời là một thuật toán phân đoạn thủ công tên là *selective search*, thứ không có tham số nào học được và không bao giờ nhận được một chút *gradient* nào từ lỗi cuối cùng.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn đọc một paper detection bất kỳ và chỉ ra ngay được ba thứ: khối nào nằm trong tầm với của gradient, khối nào là heuristic cố định, và vì thế bước cải tiến hợp lý tiếp theo là gì. Đó là một kỹ năng dự đoán, không phải kỹ năng ghi nhớ.

TRỰC GIÁC

Hãy hình dung hệ thống như một xưởng máy mà một số máy được nối vào bảng điều khiển trung tâm, còn một số thì quay tay. Máy nối vào bảng điều khiển tự chỉnh lại khi sản phẩm cuối bị lỗi; máy quay tay thì không, dù nó có gây ra lỗi hay không. “Nuốt một heuristic vào mạng” theo nghĩa cơ học chính xác là *nối thêm một máy vào bảng điều khiển*: biến một bước không khả vi, không tham số, thành một khối khả vi nhận được gradient từ hàm mất mát detection. Vì thế mỗi lần nuốt, hệ thống không chỉ nhanh hơn (bớt một công đoạn rồi) mà còn chính xác hơn (thêm một khối được tối ưu cho đúng mục tiêu cuối).

A. Bốn bước nuốt

I MẠCH 1 — detection hai giai đoạn

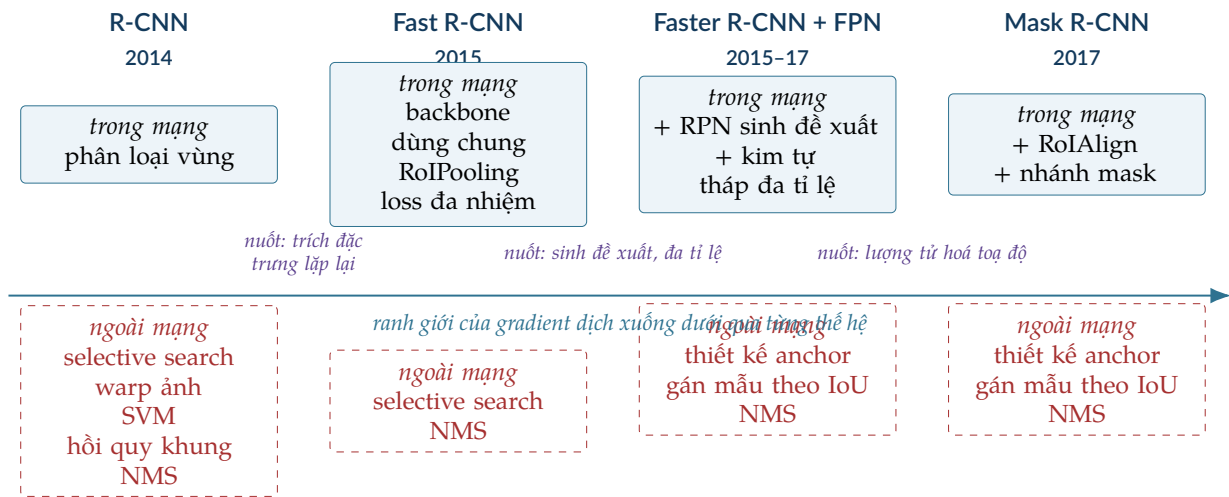
1 Vấn đề

không thể chấm điểm một tập hợp có lực lượng thay đổi.

Giải pháp

R-CNN cắt khoảng hai nghìn vùng ứng viên do *selective search* sinh ra. Mỗi vùng được warp về kích thước cố định rồi cho qua CNN để lấy đặc trưng. Một SVM phân loại, và một bộ hồi quy riêng tinh chỉnh khung [1].

- Giả định** các vùng đáng quan tâm đều nằm trong tập ứng viên; nếu selective search bỏ sót thì không gì cứu được.
- Giá phải trả** mỗi ảnh phải chạy CNN hai nghìn lần, huấn luyện chia làm ba pha rời rạc, và không pha nào tối ưu cho mục tiêu của pha kia.
- 2 Vấn đề** hai nghìn lượt chạy CNN cho cùng một ảnh là lãng phí gần như toàn bộ.
- Giải pháp** Fast R-CNN đảo thứ tự: chạy *backbone* **một lần** trên cả ảnh, rồi với mỗi vùng ứng viên chỉ cắt phần tương ứng trên feature map bằng *RoIPooling* và cho qua vài tầng **kết nối đầy đủ** (*fully connected*) [2]. Phân loại và hồi quy khung được gộp thành một **hàm mất mát đa nhiệm** (*multi-task loss*) duy nhất, huấn luyện một pha.
- ↔ **ràng buộc bởi** B/09 — trọng số giữa hạng thức phân loại và hạng thức hồi quy là một siêu tham số ẩn, và chương thiết kế hàm mất mát giải thích vì sao nó không thể chọn tùy tiện.
- Giả định** đặc trưng dùng chung cho mọi vùng vẫn đủ tốt cho từng vùng — đúng khi các vùng chồng lấn nhiều, và chúng chồng lấn thật.
- Vấn đề mới** khi phần tính toán nặng đã bị loại bỏ, selective search trở thành nút cổ chai.
- 3 Vấn đề** bộ sinh đề xuất là công đoạn quay tay cuối cùng còn nặng.
- Giải pháp** Faster R-CNN thay nó bằng **mạng đề xuất vùng** (*region proposal network, RPN*), một mạng con trượt trên chính feature map đã có, tại mỗi vị trí dự đoán độ khách quan và độ lệch khung so với một tập *anchor* định sẵn [3]. Vì RPN dùng chung backbone, chi phí sinh đề xuất gần như bằng không.
- Giả định** tập anchor phủ đủ dày dãi tỉ lệ và tỉ số cạnh của vật thể trong dữ liệu.
- Giá phải trả** anchor trở thành một **siêu tham số** (*hyperparameter*) nhạy và *mang theo thống kê của tập dữ liệu gốc*; đổi miền ảnh thì phải thiết kế lại.
- 4 Vấn đề** một feature map duy nhất không phục vụ nổi mọi tỉ lệ. Feature ở tầng sâu có ngữ nghĩa mạnh nhưng độ phân giải thô, feature nông thì ngược lại; vật nhỏ vì thế bị thiệt kép.
- Giải pháp** **kim tự tháp đặc trưng** (*feature pyramid network, FPN*) dựng một kim tự tháp bằng nhánh *top-down* cộng với kết nối ngang, để mọi tầng đều vừa sâu về ngữ nghĩa vừa đúng độ phân giải [4].
- Hệ quả** đây là lần đầu “đa tỉ lệ” được đưa vào kiến trúc thay vì đưa vào pipeline dưới dạng chạy ảnh ở nhiều kích thước.
- ↔ **tiền đề** C/13/01 — stride và trường tiếp nhận của backbone chính là thứ tạo ra mâu thuẫn ngữ nghĩa–độ phân giải mà FPN sinh ra để gỡ.
- 5 Vấn đề** RoIPooling làm tròn toạ độ, và việc làm tròn đó phá hỏng mọi bài toán cần độ chính xác dưới pixel.
- Giải pháp** Mask R-CNN thay bằng *RoIAlign*: không làm tròn, lấy mẫu song tuyến tại các điểm có toạ độ thực, rồi thêm một nhánh dự đoán **mặt nạ** (*mask*) nhị phân song song với nhánh phân loại [5].
- Hệ quả** nhánh mask gần như miễn phí về ý tưởng nhưng chỉ hoạt động tốt *sau khi* lỗi lượng tử hoá được gỡ bỏ — một minh hoạ sắc nét rằng chi tiết cài đặt có thể chặn đứng cả một bài toán mới.

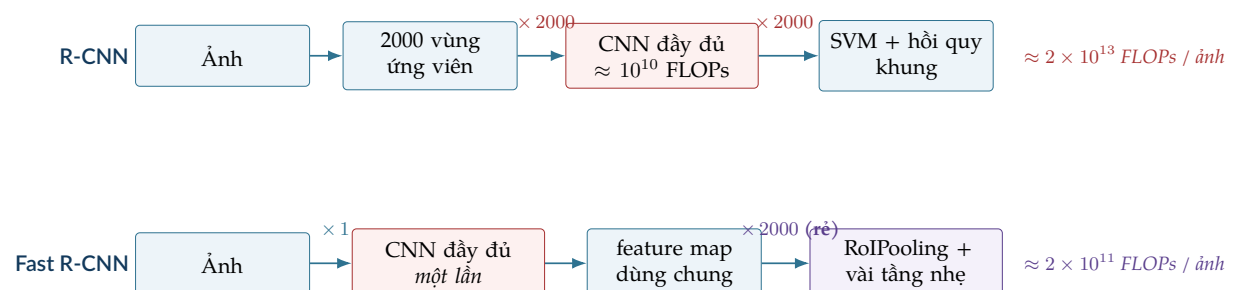


Hình 1.2. Dòng thời gian của họ hai giai đoạn, vẽ theo trục “cái gì nhận được gradient”. Khối trên đường kẻ nằm trong tầm với của hàm mất mát detection và tự chỉnh; khối dưới đường kẻ là heuristic cố định. Nhấn tím giữa hai cột ghi rõ mỗi bước nuốt được heuristic nào. Qua bốn thế hệ, phần dưới teo dần nhưng NMS và cách gán mẫu thì ở lại — đó là dư địa mà Mục 2 khai thác.

Hình 1.2 cho thấy điều mà một bảng thời gian không cho thấy: cái còn sót lại sau bốn bước không phải là ngẫu nhiên. **Triệt phi cực đại** (*non-maximum suppression, NMS*) sống sót vì nó thao tác trên *quan hệ giữa các dự đoán* chứ không trên một dự đoán đơn lẻ, mà loss của kiến trúc này lại chấm điểm từng dự đoán độc lập. Cách **gán mẫu** (*label assignment*) sống sót vì nó là một phần của *định nghĩa* loss chứ không phải một khối tính toán — không có gì để “nuốt” theo nghĩa thông thường. Hai chỗ đó đòi hỏi đối phát biểu bài toán, và đó đúng là việc DETR làm ở mục sau.

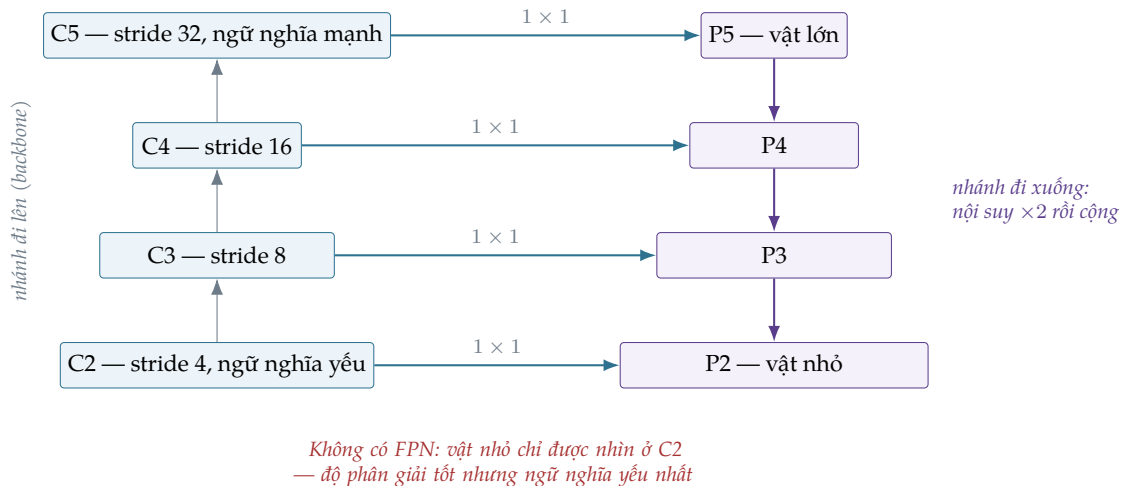
B. Ba phép tính nhỏ giải thích ba bước nhảy

- **Vì sao chia sẻ feature map lại nhanh hơn hai bậc.** Giả sử backbone tốn khoảng 10^{10} FLOPs cho một ảnh 224×224 .
 - **R-CNN** chạy nó cho từng vùng trong $N = 2000$ vùng: 2×10^{13} FLOPs mỗi ảnh.
 - **Fast R-CNN** chạy backbone một lần trên ảnh lớn hơn (giả sử đắt gấp mười, 10^{11}), rồi mỗi vùng chỉ qua vài tầng nhẹ cỡ 10^8 , tổng 2×10^{11} .
 - **Tỉ số** khoảng $2 \times 10^{13} / 3 \times 10^{11} \approx 70$ lần, và thực tế còn lớn hơn vì R-CNN phải ghi/đọc đặc trưng ra đĩa. *Ý nghĩa*: chi phí chuyển từ $O(N)$ lượt backbone sang $O(1)$ lượt backbone cộng $O(N)$ lượt đầu nhẹ — đó mới là điều cần nhớ, không phải con số.



Hình 1.3. Cùng số vùng ứng viên, cùng backbone, khác nhau ở *thứ tự* phép cắt và phép trích đặc trưng. R-CNN đặt phép cắt trước nên khối đắt nhất bị nhân với N ; Fast R-CNN đặt phép cắt sau nên khối đắt chỉ chạy một lần và chỉ phần nhẹ mới bị nhân. Toàn bộ khoảng cách hai bậc độ lớn nằm ở chỗ đổi chỗ hai mũi tên, không ở chỗ nào khác.

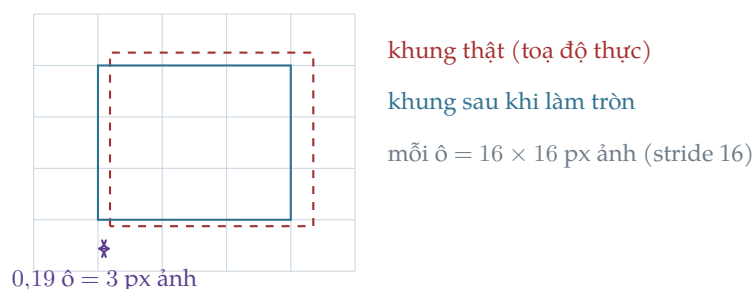
Hình 1.3 là dạng nhìn thấy được của phép tính vừa rồi, và nó cũng là mẫu hình đáng nhớ: khi một pipeline lập một khối đất, hãy hỏi liệu có đẩy được khối đó ra ngoài vòng lặp hay không. **Vì sao làm tròn lại đầu đến thế.** Backbone thông dụng có *stride* 16, nên một pixel feature ứng với ô 16×16 pixel ảnh. Khung có cạnh trái tại $x = 19$ rơi vào toạ độ feature $19/16 = 1,1875$; RoIPooling làm tròn xuống 1, tức dịch biên $0,1875 \times 16 = 3$ pixel ảnh gốc, rồi chia ô pooling làm tròn lần thứ hai. Ba pixel là nhiều với người cao 200 pixel, nhưng là 10% kích thước với một vật 30 pixel. Đó là lý do RoIAlign gần như không đổi gì ở ngưỡng **giao trên hợp** (*intersection over union, IoU*) lỏng nhưng cải thiện mạnh ở ngưỡng chặt và với nhánh mask [5].



Hình 1.4. FPN gộp hai thứ vốn nằm ở hai đầu đối lập của backbone. Đi lên, độ phân giải giảm còn ngữ nghĩa mạnh dần; đi xuống, ngữ nghĩa của tầng sâu được nội suy trả về từng mức và cộng vào đặc trưng nông qua kết nối ngang 1×1 . Kết quả là mọi mức P_i đều vừa đủ sâu vừa đủ mịn, nên vật nhỏ không còn bị thiệt kép.

- Hình 1.4 làm rõ vì sao đây là một bước “nuốt heuristic”: trước FPN, cách xử lý đa tỉ lệ là chạy *cả ảnh* ở nhiều kích thước rồi gộp kết quả — một vòng lặp bên ngoài mạng, tốn gấp nhiều lần. FPN đưa đúng chức năng đó vào trong một nhánh có tham số học được.

FPN gán vùng vào tầng nào. Công thức trong bài gốc là $k = \lfloor k_0 + \log_2(\sqrt{wh}/224) \rfloor$ với $k_0 = 4$. Nó nói gì: vật càng nhỏ thì càng được đẩy về tầng có độ phân giải cao. Khung 112×112 cho $\log_2(112/224) = -1$ nên $k = 3$; khung 448×448 cho $k = 5$. Đáng chú ý là công thức này vẫn là heuristic thủ công nằm ngoài gradient — lại một vòng nữa của cùng mẫu hình.



Hình 1.5. Vì sao RoIPooling làm hỏng bài toán mask. Trên lưới đặc trưng stride 16, việc làm tròn toạ độ khung về ô nguyên dịch biên đi tới vài pixel trên ảnh gốc; sai số này cố định, không giảm khi huấn luyện lâu hơn, và tỉ lệ nghịch với kích thước vật. RoIAlign lấy mẫu song tuyến tại toạ độ thực nên không sinh ra khoảng lệch màu tím.

Hình 1.5 là một lời nhắc quen thuộc với người làm hình học. Đây đúng là lỗi lượng tử hoá mà mọi hệ bám đặc trưng dưới pixel đều phải xử lý. Cách chữa cũng giống hệt: nội suy thay vì làm tròn, đúng như một bộ *tracker* KLT tinh chỉnh vị trí đặc trưng.

C. Giới hạn và trường hợp hỏng

Họ hai giai đoạn hỏng theo ba kiểu rất đặc trưng, và mỗi kiểu có một triệu chứng quan sát được:

- **Cảnh đông đúc.** Khi hai vật cùng lớp chồng lấn nhiều hơn ngưỡng NMS, một trong hai chắc chắn bị xoá. *Triệu chứng:* hạ ngưỡng thì mất vật, nâng ngưỡng thì sinh trùng lặp — không có giá trị nào cứu được cả hai, vì vấn đề nằm ở phát biểu chứ không ở tham số.
- **Miền anchor lệch.** Tập anchor mã hoá phân bố tỉ lệ và tỉ số cạnh của dữ liệu huấn luyện. Đưa sang ống kính góc rộng, ảnh *fisheye*, hay lớp vật dài mảnh thì recall của RPN sụt trước khi bộ phân loại kịp nói gì. *Triệu chứng:* recall giai đoạn đề xuất thấp trong khi độ chính xác trên các đề xuất còn lại vẫn cao.
- **Độ trễ không ổn định.** Số đề xuất qua ngưỡng thay đổi theo ảnh, nên thời gian giai đoạn hai dao động theo nội dung cảnh. *Triệu chứng:* phân vị 99 của độ trễ cao hơn nhiều so với trung vị — một tính chất rất khó chịu khi bạn phải cam kết ngân sách mili-giây cho mỗi khung hình, đúng kiểu ràng buộc cứng đã bàn ở B/03-giai-phau-he-thong.

CẠM BẦY

Rất nhiều so sánh “kiến trúc A tốt hơn B 2 mAP” thực chất là so sánh hai công thức huấn luyện: số epoch, **tăng cường dữ liệu** (*data augmentation*), tỉ lệ mẫu dương/âm trong mỗi *batch*, ngưỡng IoU khi gán mẫu. Trước khi tin một cải tiến kiến trúc, hãy tìm xem phần gán mẫu và recipe có được giữ nguyên không — và trong đa số paper, thông tin đó nằm trong code chứ không nằm trong bài.

↔ **hệ quả của** E/24 — muốn tách phần đóng góp của một khối khỏi phần đóng góp của recipe thì phải ablate đúng một thứ — đó là phương pháp, không phải trực giác.

Công cụ như [dbolya/tide](#) tách mAP thành các loại lỗi thành phần [6], [7], và thường phát hiện rằng hai mô hình “chênh 2 mAP” hỏng theo hai kiểu hoàn toàn khác nhau.

CẦN NHỚ

Mỗi bước tiến của detection hai giai đoạn là một heuristic bị kéo vào trong tầm với của gradient. Muốn đoán bước tiếp theo của một dòng mô hình bất kỳ, kể cả *SLAM*, hãy liệt kê những gì còn nằm *ngoài* vòng tối ưu. Rồi hỏi cái nào trong số đó viết lại được thành một khối khả vi. [T3]

D. Kết nối

Mục tiếp theo (C/14-bai-toan-cv-loi/02-detection-mot-giai-doan) là mặt đối lập: bỏ hẳn giai đoạn đề xuất để đổi lấy tốc độ, và trả giá bằng một vấn đề mất cân bằng mà mục này chưa gặp. C/13-kien-truc-backbone/01-dong-cnn là tiền đề: RoIPooling, stride và trường tiếp nhận chỉ hiểu được sau khi nắm cơ học của *convolution*. B/09-thiet-ke-ham-mat-mat giải thích vì sao gộp phân loại và hồi quy vào một loss đa nhiệm không hiển nhiên đúng. E/24-thi-nghiem-va-ablation là nơi kiểm chứng cạm bẫy vừa nêu.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao Fast R-CNN nhanh hơn R-CNN nhiều bậc trong khi cả hai dùng cùng backbone và cùng số vùng ứng viên?
2. RoIAlign khác RoIPooling ở đúng một chỗ. Vì sao chỗ đó ảnh hưởng tới nhánh mask mạnh hơn nhiều so với nhánh phân loại?
3. Miền ảnh cụ thể nào làm tập anchor mặc định của COCO sụt recall, và triệu chứng quan sát được ở đó là gì?

4. Vì sao NMS và cách gán mẫu là hai thứ duy nhất sống sót qua cả bốn thế hệ? Hai lý do đó khác nhau ở điểm nào?
5. Nếu bạn phải cam kết một ngân sách mili-giây cố định cho mỗi khung hình, kiến trúc hai giai đoạn gây khó khăn gì mà một giai đoạn không gây?

Nguồn

- [1] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.
- [2] R. Girshick, “Fast R-CNN,” in *ICCV*, 2015.
- [3] S. Ren, K. He, R. Girshick, et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *NeurIPS*, 2015.
- [4] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [5] K. He, G. Gkioxari, P. Dollár, et al., “Mask R-CNN,” in *ICCV*, 2017.
- [6] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [7] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [8] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [9] W. Liu, D. Anguelov, D. Erhan, et al., “SSD: Single Shot MultiBox Detector,” in *ECCV*, 2016.
- [10] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [11] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [12] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [13] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [14] Z. Ge, S. Liu, Z. Li, et al., “OTA: Optimal Transport Assignment for Object Detection,” in *CVPR*, 2021.
- [15] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [16] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [17] X. Zhu, W. Su, L. Lu, et al., “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *ICLR*, 2021.
- [18] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [19] H. Zhang, F. Li, S. Liu, et al., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection,” in *ICLR*, 2023.
- [20] J. Long, E. Shelhamer, and T. Darrell, “Fully Convolutional Networks for Semantic Segmentation,” in *CVPR*, 2015.
- [21] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [22] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.

- [23] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,” *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [24] E. Xie, W. Wang, Z. Yu, et al., “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers,” in *NeurIPS*, 2021.
- [25] B. Cheng, I. Misra, A. G. Schwing, et al., “Masked-Attention Mask Transformer for Universal Image Segmentation,” in *CVPR*, 2022.
- [26] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [27] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [28] K. Sun, B. Xiao, D. Liu, et al., “Deep High-Resolution Representation Learning for Human Pose Estimation,” in *CVPR*, 2019.
- [29] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [30] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [31] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [32] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.

1.2 Một giai đoạn và cuộc đánh đổi tốc độ (One-Stage Detection and the Speed Trade-off)

PHỤ THUỘC C/14-bai-toan-cv-loi/01-detection-hai-giai-doan (anchor, NMS, RoI); B/09-thiet-ke-ham-mat-mat (focal loss, mất cân bằng lớp); C/13/02-tu-chuoi-toi-attention (attention, cần cho phần DETR).

TÓM TẮT

Mục này trả lời ba câu hỏi. Vì sao bỏ giai đoạn đề xuất vùng lại làm lộ ra mất cân bằng nền–vật thể? Vì sao vấn đề đó được chữa ở tầng hàm mất mát chứ không ở tầng kiến trúc? Và vì sao chuỗi ATSS → SimOTA → Hungarian matching mới là trực tiến hoá thật của detection hiện đại? Đọc xong, trong bất kỳ bảng so sánh detector nào bạn cũng tách được phần chênh lệch *do kiến trúc* khỏi phần chênh lệch *do gán mẫu và công thức huấn luyện*. Bạn cũng biết tìm thông tin đó ở đâu khi paper không nói. Nếu chỉ nhớ một điều: cách gán mẫu cho 10^5 vị trí dự đoán là một phần của *định nghĩa bài toán tối ưu*. Hai detector gán mẫu khác nhau vì thế là hai bài toán khác nhau. So sánh chúng rồi quy kết cho kiến trúc là sai ngay từ tiền đề.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm mất mát (loss function)** — một số duy nhất đo mức sai của dự đoán so với nhãn; toàn bộ việc huấn luyện là chỉnh tham số để số đó nhỏ đi. Đối hàm mất mát là đối *mục tiêu*, không phải đối mô hình.
- **cross-entropy** — hàm mất mát chuẩn cho phân loại: $-\log p_t$ với p_t là xác suất mô hình gán cho lớp đúng. Đoán đúng chắc chắn thì mất mát gần 0; đoán sai chắc chắn thì mất mát rất lớn.
- **mẫu dương / mẫu âm** — với mỗi vị trí dự đoán, “dương” nghĩa là vị trí đó được coi là chứa vật thể và phải học nhãn của vật đó; “âm” nghĩa là nền. Ai được coi là dương là do bước gán mẫu quyết định.

- **gán mẫu (label assignment)** — quy tắc quyết định mỗi trong hàng trăm nghìn vị trí dự đoán phải học điều gì. Nó không phải một khối tính toán mà là một phần của *định nghĩa* hàm mất mát — đây là ý trung tâm của cả mục.
- **anchor và anchor-free** — có anchor là dự đoán độ lệch so với các khung mẫu định sẵn; anchor-free là dự đoán trực tiếp từ mỗi ô lưới, chẳng hạn bốn khoảng cách tới bốn cạnh của khung.
- **NMS** — hậu xử lý xoá khung trùng: giữ khung điểm cao nhất rồi xoá mọi khung chồng lấn với nó quá ngưỡng IoU. Vì nó xét quan hệ *giữa* các dự đoán nên không thể huấn luyện cùng phần còn lại.
- **truy vấn (query) trong DETR** — một vector học được đóng vai “một chỗ trống để điền một vật thể”. Mô hình có N truy vấn cố định, mỗi truy vấn tự đi hỏi feature map rồi trả về một khung cộng một nhãn.
- **epoch** — một lượt đi hết toàn bộ tập huấn luyện. “Hội tụ sau 500 epoch” nghĩa là phải quét dữ liệu 500 lần mới đạt kết quả tốt — rất đắt.

Đặt vấn đề

Giai đoạn đề xuất vùng trong Faster R-CNN giải quyết được một vấn đề rất cụ thể: nó *lọc* hàng chục nghìn vị trí ứng viên xuống còn vài trăm, và mọi thứ phía sau chỉ phải làm việc trên một tập nhỏ, đã tương đối cân bằng giữa mẫu dương và mẫu âm. Cái giá là một vòng lặp nữa qua bộ nhớ và một lượng tính toán tỉ lệ với số đề xuất. Với robot phải chạy 30 khung hình mỗi giây trên một máy nhúng, cái giá đó thường không trả nổi. Câu hỏi tự nhiên là: bỏ hẳn giai đoạn đề xuất, cho mạng dự đoán thẳng trên lưới đặc trưng, thì mất gì?

Học xong mục này bạn làm được gì mà trước đó không làm được? Đọc phần thực nghiệm của một paper detection, bạn biết ngay cần kiểm tra ba dòng nào trong code trước khi tin bằng kết quả.

YOLO và SSD trả lời bằng cách làm đúng thế [8], [9]. Kết quả là tốc độ tăng vọt, nhưng một vấn đề mới xuất hiện mà kiến trúc hai giai đoạn chưa từng phải đối mặt, bởi vì bộ lọc đề xuất đã âm thầm che nó đi: **mất cân bằng nền–vật thể** (*foreground–background imbalance*). Bài học lớn nhất của mục này nằm ở chỗ vấn đề đó được giải quyết ở tầng **hàm mất mát** (*loss function*), không phải ở tầng kiến trúc. Câu chuyện sau đó còn cho thấy một điều nữa: phần lớn khoảng cách giữa các detector hiện đại nằm ở những chi tiết *không được viết trong abstract*.

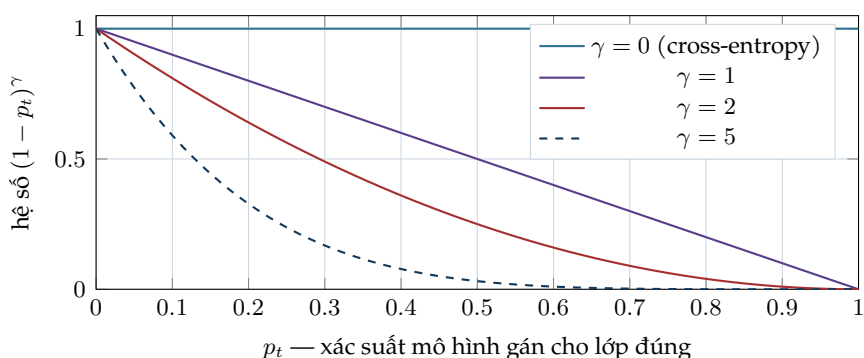
A. Mất cân bằng, và vì sao nó là vấn đề của loss (Class Imbalance)

Hãy đếm. Một mạng một giai đoạn với **kim tự tháp đặc trưng** (*feature pyramid network, FPN*) năm tầng, ảnh 800×1333 , chín **anchor** mỗi vị trí, sinh ra cỡ 10^5 dự đoán mỗi ảnh. Một ảnh COCO trung bình có bảy vật thể; kể cả khi mỗi vật khớp vài anchor thì số mẫu dương vẫn chỉ vài chục. Tỉ lệ âm trên dương do đó vào khoảng 10^3 tới 10^4 .

Con số đó chưa đủ để thấy vấn đề. Cái đáng sợ nằm ở tổng gradient. Giả sử sau vài epoch, mạng đã học được rằng gần như mọi thứ là nền. Mỗi anchor nền khi đó có xác suất đúng $p_t = 0,99$, đóng góp **entropy chéo** (*cross-entropy*) $-\log 0,99 \approx 0,01$. Nghe như không đáng kể — nhưng 10^5 anchor nền cho tổng $10^5 \times 0,01 = 1000$. Trong khi đó ba mươi anchor dương khó, mỗi cái $p_t = 0,5$ và loss 0,69, chỉ cho tổng 21. Tín hiệu học thật bị nhấn chìm dưới tỉ lệ 50:1 bởi những mẫu mà mạng đã làm đúng rồi. Đây là ví dụ nhỏ nhất giải thích vì sao “hạ learning rate” hay “train lâu hơn” không cứu được: bản thân hàm mất mát đang chỉ sai hướng.

Cách chữa cổ điển là **hard negative mining** — SSD giữ tỉ lệ âm/dương 3:1 bằng cách chỉ lấy các mẫu âm có loss cao nhất. Cách này hoạt động nhưng đưa thêm một heuristic rời rạc, không khả vi vào giữa vòng huấn luyện. **Focal loss** thay nó bằng một hệ số điều biến trơn: $FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t$ [10]. Với $\gamma = 2$, mẫu dễ $p_t = 0,99$ bị nhân với $(0,01)^2 = 10^{-4}$. Mẫu khó $p_t = 0,5$

chỉ bị nhân 0,25. Lặp lại phép đếm ở trên: tổng đóng góp của nền tảng từ 1000 xuống 0,1, trong khi mẫu dương khó chỉ giảm bốn lần. Tỷ lệ đảo ngược hoàn toàn.



Hình 1.6. Hệ số điều biến của focal loss. Điều quan trọng không phải mẫu khó được đề cao, mà là mẫu *đã đúng* bị hạ xuống gần như bằng không — với $\gamma = 2$, một anchor nền chắc chắn đóng góp ít hơn bốn bậc so với cross-entropy. Vì γ chỉ nén chứ không đảo thứ tự, nó không thể sửa được một tập nhãn sai.

Hình 1.6 cũng cho thấy giới hạn. Hệ số luôn dương và đơn điệu. Nên khi một mẫu bị gán nhãn sai, mô hình dự đoán *đúng theo thực tế nhưng sai theo nhãn*, focal loss coi nó là mẫu khó và *tăng* trọng số cho nó. Trên dữ liệu nhiễu loạn, đây là chế độ hỏng thật, không phải lý thuyết.

→ **trường hợp riêng của B/09** — focal loss là một ca cụ thể của nguyên tắc tổng quát “sửa mất cân bằng ở tầng hàm mất mát thay vì ở tầng lấy mẫu dữ liệu”.

Kết luận rút ra từ RetinaNet vượt xa bản thân hàm loss. Suốt vài năm, khoảng cách giữa một giai đoạn và hai giai đoạn được giải thích bằng “kiến trúc hai giai đoạn tinh vi hơn”. Hoá ra phần lớn khoảng cách đó là hệ quả của việc giai đoạn đề xuất đã ngầm cân bằng lại tập mẫu. Sửa ở tầng loss là đủ.

B. Bỏ anchor, và câu hỏi thật sự là gán mẫu (Label Assignment)

TRỰC GIÁC

Kiến trúc là thí sinh; cách gán mẫu là *đáp án*. Một detector một giai đoạn có 10^5 ô trên lưới, và gán mẫu chính là việc điền vào đáp án cho từng ô: ô này phải nói “có xe ở đây”, ô kia phải nói “không”. Đối cách gán mẫu không phải là đối thí sinh, mà là đối đáp án — tức đối hần bài toán tối ưu. Vì thế so hai detector dùng hai cách gán mẫu khác nhau rồi kết luận về kiến trúc cũng vô nghĩa như so điểm hai thí sinh làm hai đề khác nhau.

I MẠCH 2 — một giai đoạn, từ anchor tới tập hợp

1 Vấn đề

Giải pháp

Giá phải trả

anchor là một siêu tham số thủ công mang theo thống kê của tập dữ liệu.

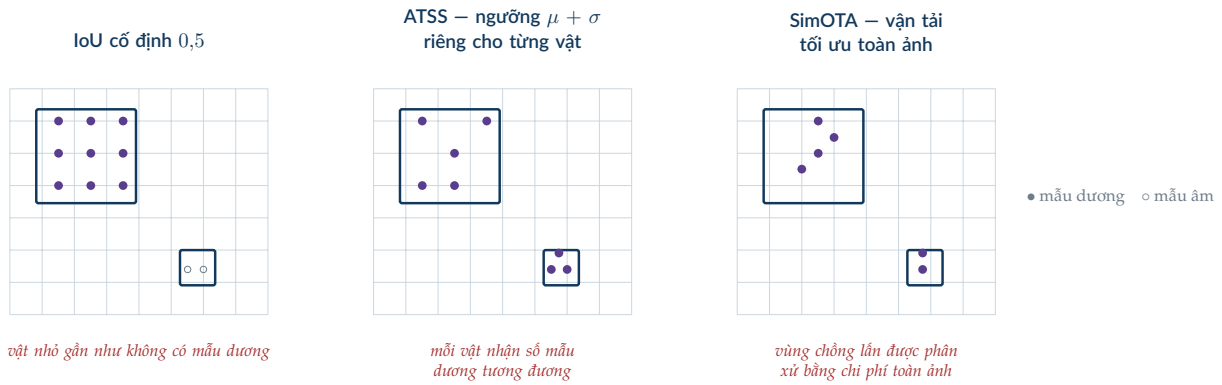
họ *anchor-free* dự đoán trực tiếp từ mỗi vị trí trên lưới: FCOS dự đoán bốn khoảng cách từ điểm đó tới bốn cạnh của khung [11], CenterNet dự đoán một heatmap tâm vật cộng kích thước [12].

không còn tập anchor, nhưng bây giờ phải trả lời một câu hỏi mới và khó hơn: *vị trí nào trên lưới chịu trách nhiệm cho vật nào*. Bài toán không biến mất, nó đổi tên thành gán mẫu.

2 Vấn đề

gán mẫu theo ngưỡng IoU cố định là một lựa chọn tùy tiện. Ngưỡng 0,5 khiến vật nhỏ chỉ nhận được một hai mẫu dương trong khi vật lớn nhận hàng chục, tức mô hình được huấn luyện với mức giám sát rất khác nhau tùy kích thước.

Giải pháp	<i>ATSS</i> — viết tắt của <i>adaptive training sample selection</i> — chọn ngưỡng thích nghi cho từng vật: lấy k ứng viên gần tâm nhất ở mỗi tầng, tính trung bình và độ lệch chuẩn IoU của chúng, dùng $\mu + \sigma$ làm ngưỡng [13].
Hệ quả	bài báo cho thấy sau khi thống nhất cách gán mẫu, khác biệt giữa detector có anchor và không anchor gần như biến mất — một trong những kết quả ablation quan trọng nhất của cả lĩnh vực.
3 Vấn đề	gán mẫu từng vật độc lập là tối ưu cục bộ. Khi hai vật chồng lấn, một vị trí lưới có thể tốt cho cả hai, và quyết định tham lam cho từng vật riêng lẻ dẫn tới xung đột.
Giải pháp	<i>optimal transport assignment</i> (OTA) và bản rút gọn SimOTA phát biểu việc gán mẫu thành một bài toán vận tải tối ưu: mỗi ground truth là một nguồn cung k nhãn dương, mỗi vị trí lưới là một điểm cầu, chi phí là tổ hợp loss phân loại và loss hồi quy [14].
Giá phải trả	thêm chi phí tính toán trong lúc huấn luyện, không thêm gì lúc suy luận.
4 Vấn đề	NMS là heuristic cuối cùng, và nó là heuristic duy nhất thao tác trên quan hệ giữa các dự đoán. Soft-NMS làm mềm bằng cách giảm điểm thay vì xoá hẳn [15], nhưng vẫn là hậu xử lý.
Giải pháp	DETR đổi phát biểu: dự đoán một tập hợp cố định N truy vấn, ghép chúng với chân trị (<i>ground truth</i>) bằng ghép cặp Hungary (<i>Hungarian matching</i>) một-một, và loss chỉ được định nghĩa sau khi ghép [16]. Đây là ý tưởng dự đoán tập hợp (<i>set prediction</i>) mà Mục 3 sẽ mượn nguyên vẹn sang segmentation. Vì mỗi vật chỉ có đúng một dự đoán được thưởng, mô hình học cách tự triệt tiêu trùng lặp; NMS biến mất khỏi pipeline.
Giá phải trả	hội tụ cực chậm — bản gốc cần khoảng 500 epoch trong khi Faster R-CNN quen dùng lịch 12–36 epoch — và kém trên vật nhỏ.
5 Vấn đề	vì sao DETR chậm hội tụ? Hai nguyên nhân được các bản sửa chỉ ra: attention dày đặc trên toàn feature map khởi đầu gần như đồng đều nên mất rất lâu để tập trung, và phép ghép một-một dao động giữa các epoch nên tín hiệu giám sát cho mỗi truy vấn không ổn định.
Giải pháp	Deformable DETR cho mỗi truy vấn chỉ lấy mẫu một số ít điểm quanh một điểm tham chiếu học được, giảm cả chi phí lẫn thời gian hội tụ xuống cỡ 50 epoch [17], [18]; DINO thêm truy vấn khởi nhiễu (cho mạng các khung ground truth đã bị nhiễu và bắt khôi phục) để ổn định hoá tín hiệu [19].
Hệ quả	tới 2026, dòng thực dụng như RT-DETR và RF-DETR đã đạt được cả tốc độ lẫn độ chính xác mà không cần NMS.



Hình 1.7. Cùng một ảnh, cùng một mạng, cùng một tập vị trí dự đoán — ba quy tắc gán mẫu cho ba tập mẫu dương khác nhau, và do đó ba hàm mất mát khác nhau. Khung đậm là hai vật thật, một lớn một nhỏ. Với ngưỡng IoU cố định, vật lớn nuốt gần hết ngân sách giám sát còn vật nhỏ hầu như không được dạy; ATSS cân lại bằng ngưỡng tính riêng cho từng vật; SimOTA thêm một bước phân xử toàn ảnh nên vị trí tranh chấp không bị gán cho cả hai. Vị trí các chấm ở đây là minh hoạ nguyên lý, không phải kết quả chạy thật.

Hình 1.7 cho thấy thứ mà một bảng số không cho thấy: cái thay đổi giữa ba cột không phải mô hình mà là *đề bài* đưa cho mô hình.

Bảng 1.1. Bốn cách trả lời câu hỏi “vị trí nào chịu trách nhiệm cho vật nào”. Đây là thành phần hiếm khi nằm trong abstract nhưng thường giải thích phần lớn chênh lệch giữa hai detector.

Cách gán	Cơ chế	Giả định	Hỏng khi nào
IoU cố định	Anchor nào có IoU vượt ngưỡng thì là dương	Ngưỡng tốt cho mọi kích thước và tỉ lệ	Vật nhỏ nhận rất ít mẫu dương; đổi phân bố kích thước là phải chỉnh lại tay
Centerness / FCOS	Điểm trong vùng trung tâm của khung là dương, có trọng số theo độ lệch tâm	Tâm hình học là điểm giàu thông tin nhất	Vật hình khuyên, hình chữ L, hoặc bị che ở giữa — tâm rơi vào nền
ATSS	Ngưỡng $\mu + \sigma$ tính riêng cho từng vật từ k ứng viên gần nhất	Phân bố IoU của ứng viên là đơn đỉnh	Cảnh rất đông: các ứng viên “gần nhất” thuộc về vật khác
OTA / SimOTA	Vận tải tối ưu toàn ảnh trên chi phí phân loại + hồi quy	Chi phí ghép phản ánh đúng chất lượng dự đoán	Nhảy với chi phí khởi đầu khi mô hình còn kém; tốn thêm thời gian huấn luyện

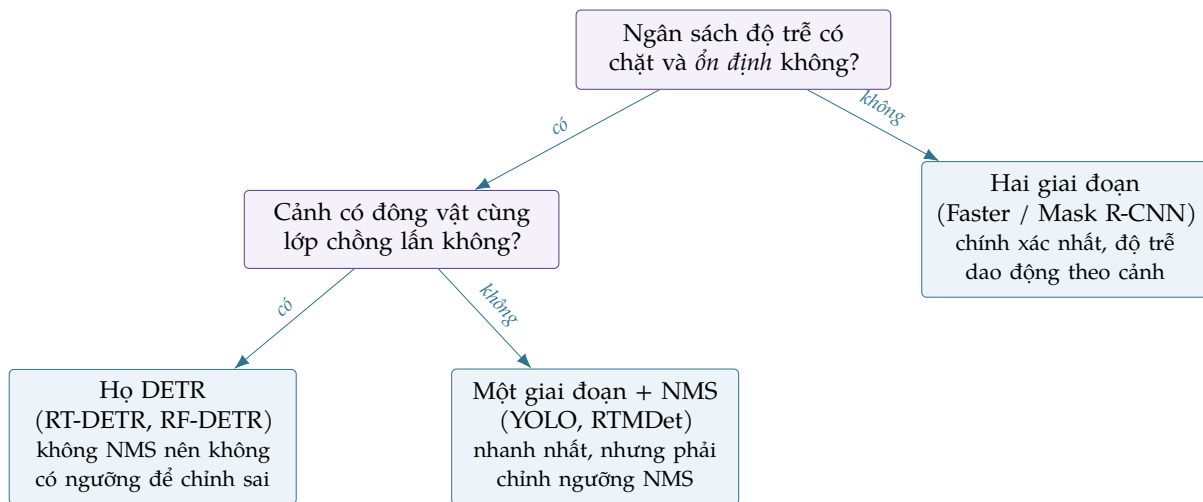
Bảng 1.1 nên đọc cùng với một quan sát khó chịu: bốn dòng này thay đổi *hàm mất mát*, không thay đổi mô hình. Hai detector khác nhau ở dòng gán mẫu là hai bài toán tối ưu khác nhau, nên so sánh chúng rồi quy kết chênh lệch cho “kiến trúc” là một suy luận sai ngay từ tiền đề.

→ cùng nguyên lý với C/14/03 — Mask2Former mượn nguyên phép ghép cặp một-một này sang miền mask, và nhờ đó hợp nhất được semantic, instance và panoptic.

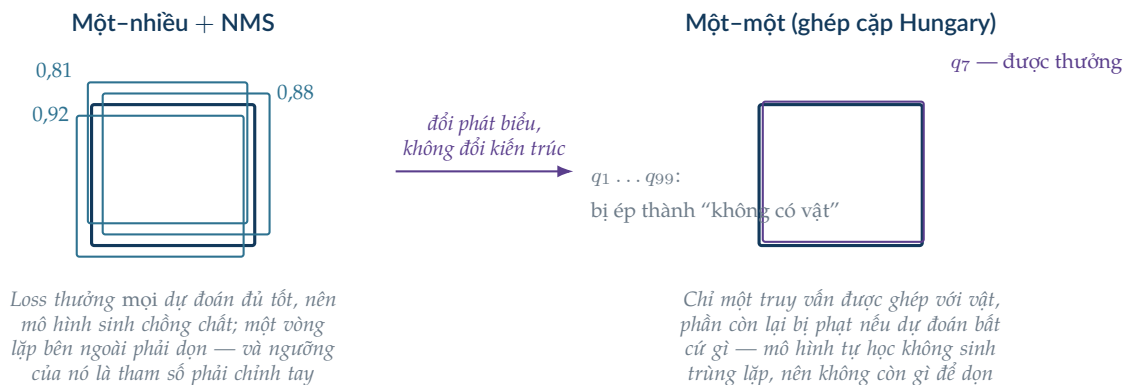
Hungarian matching của DETR, nhìn qua lăng kính này, chỉ là dòng thứ năm — một cách gán mẫu một-một thay vì một-nhiều — và chính điều đó, chứ không phải Transformer, mới là thứ khiến NMS trở nên thừa.

CẠM BÃY

Điểm số mà NMS dùng để xếp hạng là điểm *phân loại*, nhưng thứ ta cần giữ lại là khung *định vị* tốt nhất. Hai đại lượng này tương quan yếu hơn ta tưởng, nên một khung tự tin về nhãn nhưng lệch vị trí vẫn có thể xoá mất một khung đúng hơn. Các bản vá là dự đoán thêm một nhánh ước lượng IoU hoặc centerness rồi nhân vào điểm; nhưng nếu bạn thấy mAP ở ngưỡng IoU chặt (AP_{75}) tụt nhiều hơn ở ngưỡng lỏng (AP_{50}), hãy nghi ngờ đúng chỗ này trước khi đổi backbone.



Hình 1.8. Cây quyết định thực dụng: ràng buộc loại bỏ phương án trước khi độ chính xác kịp lên tiếng. Hai nhánh trái đều là một giai đoạn, nhưng khác nhau ở chỗ NMS còn hay không — và trong cảnh đồng đúc, đó là khác biệt quyết định chứ không phải vài điểm mAP.



Hình 1.9. Vì sao NMS biến mất khi đổi cách ghép. Bên trái, hàm mất mát chấm điểm từng dự đoán độc lập nên nhiều dự đoán cùng đúng và cùng được thưởng; việc chọn một trong số đó phải làm ở hậu xử lý, bằng một heuristic có ngưỡng. Bên phải, phép ghép một-một biến “sinh trùng lặp” thành hành vi bị phạt ngay trong hàm mất mát, nên bước dọn không còn lý do tồn tại. Các số tin cậy chỉ để minh họa.

Hình 1.9 cũng giải thích cái giá: vì mỗi vật chỉ có đúng một truy vấn được thưởng, tín hiệu giám sát cho mỗi truy vấn thưa hơn hẳn, và phép ghép còn dao động giữa các epoch — hai lý do khiến DETR bản gốc hội tụ chậm.

↪ **ràng buộc bởi** B/03 — cây quyết định này là hiện thân của nguyên tắc đọc hệ thống: ràng buộc cứng loại phương án trước, độ chính xác chỉ lên tiếng sau.

Hình 1.8 tóm lại mục này dưới dạng dùng được ngay: câu hỏi đầu tiên không phải “mô hình nào mAP cao nhất” mà “ràng buộc nào của tôi là cứng” — đúng cách đọc hệ thống ở B/03-giai-phau-he-thong.

C. Giới hạn và trường hợp hỏng

Detector một giai đoạn hỏng ở ba chỗ đáng nhớ, mỗi chỗ có một triệu chứng quan sát được:

- **Nhân nhiễu gặp focal loss.** Focal loss giả định nhân sạch; trên dữ liệu gán bằng đám đông hoặc bán tự động, nó khuếch đại đúng những mẫu bị gán sai và có thể tệ hơn cross-entropy thường. *Triệu chứng:* loss huấn luyện đứng yên trong khi một nhóm rất nhỏ mẫu chiếm gần hết *gradient*.
- **Gán nhãn tinh vi ở giai đoạn đầu.** SimOTA và họ hàng dựa vào chi phí ghép, mà chi phí đó vô nghĩa khi mô hình còn kém, nên chúng phải kèm một giai đoạn khởi động. *Triệu chứng:* phân kỳ hoặc hội tụ về nghiệm suy biến trong vài epoch đầu — và chi tiết khắc phục nằm trong code chứ không trong bài.
- **Ngân sách truy vấn cố định của DETR.** Ảnh chứa nhiều vật hơn N thì phần dư bị mất im lặng, không có cảnh báo nào. *Triệu chứng:* recall giảm đều theo mật độ vật thể; không lộ ra trên COCO nhưng lộ ra ngay khi bạn đếm người trong đám đông.

Với người làm hệ thống thời gian thực, còn một giới hạn nữa ít được nói: bỏ NMS không có nghĩa là bỏ hậu xử lý không xác định. Một giai đoạn có thời gian chạy gần như cố định theo nội dung cảnh, đó là ưu điểm lớn nhất của nó so với hai giai đoạn; nhưng nếu bạn thêm lại NMS trên đầu ra (như hầu hết bản triển khai YOLO làm), thời gian NMS lại tỉ lệ với số khung sống sót, và phương sai độ trễ quay về.

CẦN NHỚ

Trước khi tin bất kỳ so sánh detector nào, hãy hỏi hai câu: hai bên dùng cách gán mẫu nào, và hai bên chạy bao nhiêu epoch với augmentation gì. Rất nhiều “cải tiến kiến trúc” công bố thực chất là cải tiến gán mẫu, và nó bị chôn trong code. [T3]

D. Kết nối

C/14-bai-toan-cv-loi/01-detection-hai-giai-doan là tiền đề trực tiếp: mất cân bằng chỉ lộ ra khi bỏ giai đoạn đề xuất, nên mục này chỉ có nghĩa sau mục kia. B/09-thiet-ke-ham-mat-mat là nơi focal loss được đặt cạnh các cách xử lý mất cân bằng khác, và là nơi trả lời câu hỏi tổng quát “khi nào nên sửa ở loss thay vì sửa ở dữ liệu”. Ý tưởng dự đoán tập hợp bằng Hungarian matching quay lại nguyên vẹn ở C/14-bai-toan-cv-loi/03-segmentation dưới dạng khung truy vấn của Mask2Former — đó là trường hợp hiếm hoi một ý tưởng chuyển miền mà không mất gì. E/24-thi-nghiem-va-ablation cung cấp phương pháp để tự kiểm chứng khẳng định trung tâm của mục này.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao mất cân bằng foreground–background hầu như không xuất hiện trong Faster R-CNN nhưng lại nghiêm trọng trong SSD, dù cả hai dùng cùng anchor?
2. Hệ số điều biến với $\gamma = 2$ bằng bao nhiêu tại $p_t = 0,9$ và tại $p_t = 0,3$? Hai con số đó giải thích thế nào việc focal loss nguy hiểm trên dữ liệu nhiễu nhãn?
3. ATSS cho thấy điều gì về câu hỏi “anchor hay anchor-free”? Kết quả đó thay đổi cách bạn đọc một paper detection như thế nào?

4. DETR bỏ được NMS nhờ Hungarian matching chứ không nhờ Transformer. Hãy giải thích tại sao, và điều gì sẽ xảy ra nếu ta ghép một–nhiều thay vì một–một?
5. Bạn đo được AP_{50} cao nhưng AP_{75} thấp — hai giả thuyết nào giải thích được, và thí nghiệm nào phân biệt chúng?
6. Trên một robot có ngân sách 20 ms mỗi khung hình, vì sao phương sai độ trễ có thể quan trọng hơn độ trễ trung bình?

Nguồn

- [1] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.
- [2] R. Girshick, “Fast R-CNN,” in *ICCV*, 2015.
- [3] S. Ren, K. He, R. Girshick, et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *NeurIPS*, 2015.
- [4] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [5] K. He, G. Gkioxari, P. Dollár, et al., “Mask R-CNN,” in *ICCV*, 2017.
- [6] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [7] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [8] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [9] W. Liu, D. Anguelov, D. Erhan, et al., “SSD: Single Shot MultiBox Detector,” in *ECCV*, 2016.
- [10] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [11] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [12] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [13] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [14] Z. Ge, S. Liu, Z. Li, et al., “OTA: Optimal Transport Assignment for Object Detection,” in *CVPR*, 2021.
- [15] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [16] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [17] X. Zhu, W. Su, L. Lu, et al., “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *ICLR*, 2021.
- [18] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [19] H. Zhang, F. Li, S. Liu, et al., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection,” in *ICLR*, 2023.
- [20] J. Long, E. Shelhamer, and T. Darrell, “Fully Convolutional Networks for Semantic Segmentation,” in *CVPR*, 2015.
- [21] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [22] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.

- [23] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,” *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [24] E. Xie, W. Wang, Z. Yu, et al., “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers,” in *NeurIPS*, 2021.
- [25] B. Cheng, I. Misra, A. G. Schwing, et al., “Masked-Attention Mask Transformer for Universal Image Segmentation,” in *CVPR*, 2022.
- [26] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [27] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [28] K. Sun, B. Xiao, D. Liu, et al., “Deep High-Resolution Representation Learning for Human Pose Estimation,” in *CVPR*, 2019.
- [29] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [30] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [31] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [32] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.

1.3 Phân vùng ảnh và sự hợp nhất (Segmentation and Unification)

PHỤ THUỘC C/13-kien-truc-backbone/01-dong-cnn (stride, trường tiếp nhận (*receptive field*), upsampling); C/14-bai-toan-cv-loi/02-detection-mot-giai-doan (dự đoán tập hợp bằng Hungarian matching — cần cho phần Mask2Former).

TÓM TẮT

Mục này theo một mâu thuẫn duy nhất: mạng phân loại giỏi nhờ vứt bỏ thông tin không gian, còn phân vùng lại cần đúng thông tin vừa bị vứt. Bốn kiến trúc lớn — FCN, U-Net, DeepLab, SegFormer — là bốn cách khác nhau để lấy lại thông tin đó, mỗi cách trả một loại giá khác nhau; Mask2Former thì không lấy lại mà đổi hẳn phát biểu, và nhờ vậy hợp nhất được semantic, instance và panoptic. Đọc xong bạn chọn được kiến trúc phân vùng từ ràng buộc (bộ nhớ, lượng nhân, yêu cầu về biên) thay vì từ bảng xếp hạng, và đọc được mIoU với đúng mức hoài nghi. Nếu chỉ nhớ một điều: mọi kiến trúc segmentation đều là một câu trả lời cho “chi tiết không gian đã mất ở đâu, lấy lại bằng cách nào” — và mIoU cao không hề bảo đảm biên đúng hay vật nhỏ không bị bỏ sót.

KIẾN THỨC CẦN CÓ TRƯỚC

- **semantic segmentation** — gán cho mỗi *pixel* một nhãn lớp. Hai chiếc xe cạnh nhau đều là “xe” và không được tách ra.
- **instance segmentation** — ngoài nhãn lớp còn tách từng cá thể: xe thứ nhất và xe thứ hai là hai mask khác nhau.
- **panoptic segmentation** — gộp cả hai: các lớp đếm được thì tách cá thể, các lớp không đếm được (bầu trời, mặt đường) thì chỉ cần nhãn lớp.
- **mask** — một lưới nhị phân cùng kích thước ảnh, đánh dấu pixel nào thuộc về vùng đang nói tới. Đây là “dạng đầu ra” của mọi bài toán trong mục này.

- **output stride** — tỉ lệ thu nhỏ giữa ảnh và feature map. Output stride 32 nghĩa là ảnh 1024×1024 còn lại lưới 32×32 , nên mỗi ô đại diện cho 32×32 pixel và biên chỉ định vị được tới độ thô đó.
- **encoder-decoder** — encoder là phần đi xuống, thu nhỏ dần độ phân giải để lấy ngữ nghĩa; decoder là phần đi lên, khôi phục độ phân giải. *Skip connection* là đường nối tắt từ một tầng encoder sang tầng decoder cùng cỡ.
- **trường tiếp nhận (receptive field)** — vùng ảnh gốc mà một ô đặc trưng “nhìn thấy”. Muốn hiểu ngữ cảnh rộng thì cần trường tiếp nhận lớn, mà cách rẻ nhất để mở rộng nó lại là thu nhỏ độ phân giải — đó chính là mâu thuẫn của mục này.
- **mIoU** — thước đo chuẩn của segmentation: với mỗi lớp lấy diện tích giao chia diện tích hợp giữa vùng dự đoán và vùng thật, rồi trung bình trên các lớp.

Đặt vấn đề

Detection trả lời “có cái gì, ở khoảng nào”. Nhiều ứng dụng cần chặt hơn: một khung chữ nhật quanh một con đường không nói được đâu là mặt đường có thể đi. Segmentation đẩy độ phân giải của câu trả lời xuống tận pixel, và ngay lập tức va vào một mâu thuẫn mà cả một thập kỷ nghiên cứu xoay quanh: mạng phân loại tốt *nhờ* vứt bỏ thông tin không gian — pooling và stride là cơ chế tạo ra bất biến tịnh tiến — trong khi segmentation lại cần đúng thông tin không gian vừa bị vứt bỏ. Ta muốn ngữ nghĩa sâu và định vị chính xác cùng lúc, còn kiến trúc phân loại được thiết kế để đánh đổi cái sau lấy cái trước.

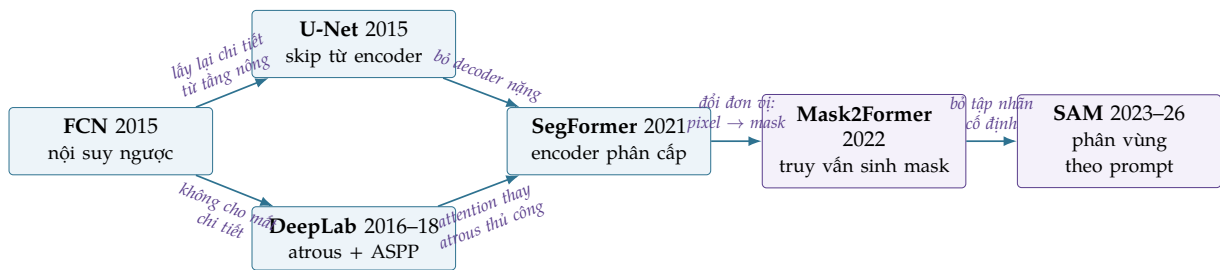
Một phép tính nhỏ cho thấy mâu thuẫn này cụ thể tới mức nào. Backbone phân loại điển hình có **output stride** 32: ảnh 1024×1024 ra feature map 32×32 . Mỗi ô đặc trưng đại diện cho một mảng 32×32 pixel. Dù bạn nội suy lên lại bằng phép gì, thông tin để phân biệt hai pixel kề nhau bên trong cùng một ô đã không còn tồn tại; sai số định vị biên có sàn cỡ chục pixel. Toàn bộ mạch dưới đây là các cách khác nhau để phá cái sàn đó, và mỗi cách trả một loại giá khác nhau.

Học xong mục này bạn làm được gì mà trước đó không làm được? Cho một yêu cầu cụ thể — “bám mép đường tới 2 pixel”, “chạy trên Jetson với 4 GB”, “chỉ có 200 ảnh gán nhãn” — bạn suy ra được kiến trúc nào khả thi và vì sao, trước khi chạy thí nghiệm nào.

TRỰC GIÁC

Semantic segmentation cổ điển hỏi mỗi pixel: “bạn thuộc lớp nào?” — một trăm nghìn câu hỏi độc lập, và mô hình không có chỗ nào để nói “ba nghìn pixel này cùng thuộc *một* vật”. Mask2Former đảo chiều điểm danh: thay vì hỏi từng pixel, nó đưa ra một trăm cái tên (truy vấn) và bắt mỗi cái tên tự nhận “tôi là vùng nào, và tôi là cái gì”. Đảo chiều câu hỏi là lý do duy nhất khiến semantic, instance và panoptic gộp được vào một mô hình: một vật thể trở thành một đơn vị được nói tới, thay vì một tính chất phát sinh từ hậu xử lý.

A. Bốn cách phá sản độ phân giải, rồi một lần đổi phát biểu (From FCN to Mask2Former)



Hình 1.10. Dòng thời gian phân vùng ảnh, đọc theo nút thắt được gỡ ở mỗi bước. Hai nhánh giữa (U-Net và DeepLab) tấn công *cùng một* vấn đề mất chi tiết từ hai phía đối lập nên có thể kết hợp; hai bước cuối không gỡ nút thắt kỹ thuật mà đổi phát biểu bài toán — đơn vị được nói tới chuyển từ pixel sang mask, rồi tập nhãn cố định bị bỏ hẳn.

Hình 1.10 đáng nhìn trước khi đọc mạch bên dưới, vì nó cho thấy ngay một điều mà đọc tuần tự khó thấy: bốn bước đầu là kỹ thuật, hai bước cuối là đổi phát biểu.

I Mạch 3 — segmentation và sự hợp nhất

1 Vấn đề

mạng phân loại không sinh được đầu ra theo pixel.

Giải pháp

FCN vứt các tầng fully connected, coi chúng như conv 1×1 , rồi dùng transposed conv để nâng độ phân giải trở lại kích thước ảnh [20].

Hệ quả

lần đầu tiên toàn bộ pipeline là một mạng duy nhất huấn luyện đầu-cuối.

Vấn đề mới

biên rất thô, vì chi tiết đã mất ở đường xuống chứ không phải ở đường lên — nội suy không thể tạo lại thứ đã bị bỏ.

2 Vấn đề

chi tiết mất ở đường xuống thì phải lấy lại từ đường xuống.

Giải pháp

U-Net nối thẳng feature ở mỗi mức của encoder sang mức tương ứng của decoder [21]. Feature nông giữ biên sắc, feature sâu giữ ngữ nghĩa, decoder hợp nhất hai loại.

Giá định

biên trong ảnh nông và biên ngữ nghĩa trùng nhau — đúng với ảnh y tế và ảnh vệ tinh, kém đúng khi vật có hoa văn mạnh bên trong.

Hệ quả

đây vẫn là kiến trúc mặc định của ảnh y tế sau mười năm, và [MIC-DKFZ/nnUNet](#) cho thấy một U-Net cấu hình tự động vẫn rất khó bị đánh bại.

3 Vấn đề

một hướng tấn công khác: đừng để mất chi tiết ngay từ đầu.

Giải pháp

tích chập giãn (*atrous / dilated convolution*) chèn khoảng trống giữa các trọng số, mở rộng trường tiếp nhận mà không giảm độ phân giải [22]; DeepLab dùng nó để giữ output stride ở mức 8 hoặc 16, và ghép nhiều tỉ lệ giãn song song thành **ASPP** [23].

Giá phải trả

bộ nhớ và tính toán, vì kích hoạt trung gian to gấp bốn lần mỗi khi giảm output stride một nửa ở cả hai chiều.

Vấn đề mới

hiệu ứng lưới (gridding) khi các tỉ lệ giãn có ước chung, làm mất mẫu ở một số vị trí.

4 Vấn đề

decoder nặng và thiết kế đa tỉ lệ thủ công.

Giải pháp

SegFormer dùng encoder Transformer phân cấp sinh sẵn đặc trưng ở bốn tỉ lệ, rồi ghép chúng bằng một decoder chỉ gồm vài tầng MLP [24].

Hệ quả

attention lo phần ngữ cảnh xa mà trước đây phải mua bằng atrous, nên decoder được phép rẻ.

Giá định

đủ dữ liệu để Transformer bù lại phần inductive bias đã bỏ.

↔ **ràng buộc bởi** C/13/03 — bỏ bớt thiên lệch quy nạp thì phải mua lại bằng dữ liệu — đó là lý do SegFormer đòi dữ liệu hơn U-Net chứ không phải vì nó “phức tạp hơn”.

5 Vấn đề

Giải pháp

semantic, instance và panoptic là ba bài toán với ba code base, ba loss, ba metric. Mask2Former phát biểu cả ba như **phân loại mặt nạ** (*mask classification*): một tập cố định **truy vấn** (*query*), mỗi truy vấn sinh ra một mask nhị phân cộng một nhãn lớp, ghép với ground truth bằng Hungarian matching — đúng ý tưởng dự đoán tập hợp của DETR, chuyển sang miền mask [25], [26].

↔ **tiền đề** C/14/02 — khung truy vấn ở đây chỉ là dự đoán tập hợp của DETR chuyển sang miền mask; chưa hiểu vì sao ghép cặp một-một bỏ được NMS thì không hiểu được vì sao ba bài toán hợp nhất được.

Đóng góp kỹ thuật then chốt là **attention có mặt nạ** (*masked attention*): mỗi truy vấn chỉ chú ý vào vùng mà nó đang dự đoán, khiến hội tụ nhanh hơn nhiều so với attention toàn cục.

Hệ quả

ba bài toán chỉ khác nhau ở cách hậu xử lý tập mask, không khác nhau ở mô hình. Khi ba bài toán hợp nhất được, thường là dấu hiệu ta đã tìm ra phát biểu đúng.

6 Vấn đề

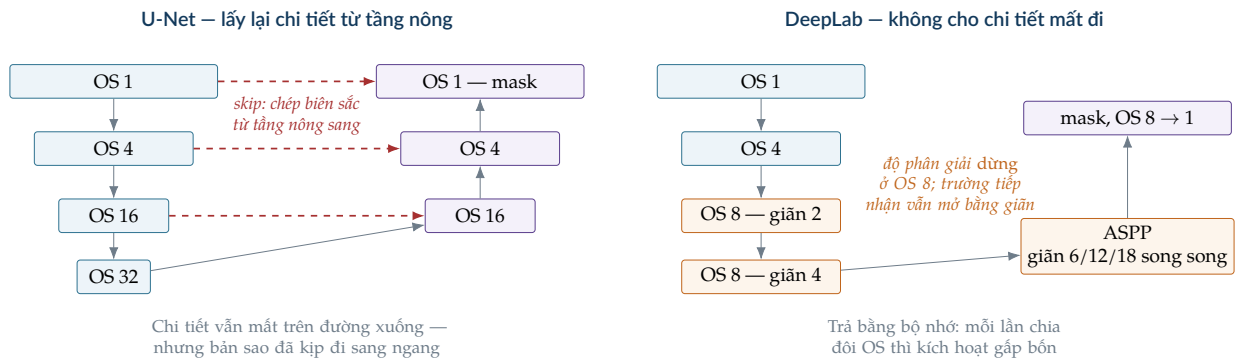
Giải pháp

tập nhãn cố định là ràng buộc cuối cùng. Mọi mô hình ở trên chỉ biết những lớp có trong tập huấn luyện.

mô hình nền tảng nhận prompt — SAM phân vùng theo điểm, khung hay vùng do người chỉ, không cần biết vật đó tên gì [27]; các hệ open-vocabulary như Grounding DINO nhận mô tả văn bản; tới 2026, SAM 3 nhận prompt ở mức khái niệm và chạy được cả trên video.

Giá phải trả

mô hình lớn, khó kiểm soát hành vi ở biên, và điều quan trọng với kỹ sư: đầu ra không kèm một khái niệm “lớp” ổn định để hệ thống phía sau dựa vào.

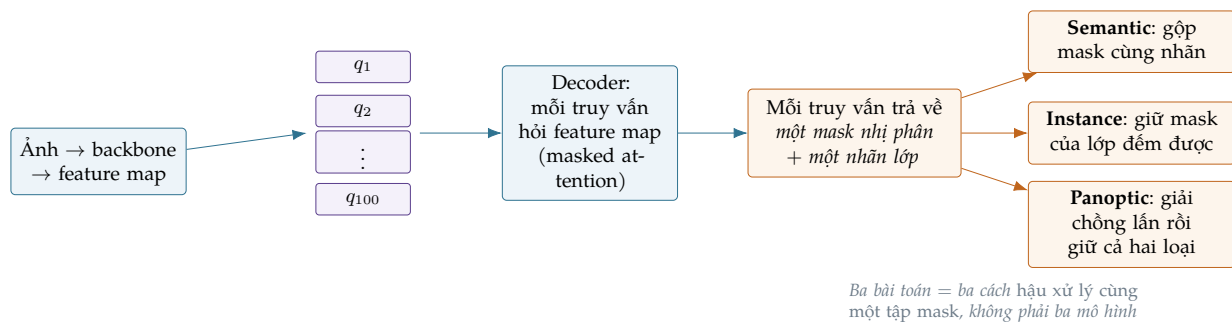


Hình 1.11. Hai lời giải đối lập cho cùng một vấn đề. U-Net chấp nhận để chi tiết mất trên đường xuống rồi chép nó sang ngang bằng skip connection; DeepLab từ chối để mất, giữ output stride (OS) ở mức 8 và mua trường tiếp nhận bằng tích chập giãn thay vì bằng thu nhỏ. Vì hai cơ chế độc lập nhau nên chúng cộng được: nhiều kiến trúc thực dụng dùng cả hai.

Hình 1.11 là lý do nên đọc mạch trên theo trục “chi tiết mất ở đâu” chứ không theo trục thời gian: hai bước liền nhau trong lịch sử lại là hai hướng tấn công khác nhau, không phải cái sau thay thế cái trước.

Bảng 1.2. Năm cách lấy lại độ phân giải, cùng cái giá và chế độ hỏng riêng. Ba dòng đầu tấn công cùng một vấn đề từ ba phía khác nhau, nên chúng có thể kết hợp; dòng cuối đối phát biểu chứ không đối cơ chế.

Cách	Cơ chế	Mạnh khi nào	Hỏng khi nào
FCN	Nội suy ngược từ feature sâu	Vùng lớn, biên không quan trọng	Biên thô; vật mảnh (dây, cột) biến mất hẳn
U-Net	Skip connection encoder → decoder	Ít dữ liệu, biên rõ, ảnh y tế	Vật có kết cấu nội bộ mạnh: skip mang cả nhiễu lên
Atrous / ASPP	Giữ output stride nhỏ, mở trường tiếp nhận bằng giãn	Cần ngữ cảnh xa mà vẫn cần biên	Bộ nhớ tăng mạnh; gridding nếu tỉ lệ giãn không nguyên tố cùng nhau
SegFormer	Encoder phân cấp + decoder MLP	Dữ liệu nhiều, cần ngữ cảnh toàn ảnh	Dữ liệu ít; chi phí attention ở độ phân giải cao
Mask2Former	Truy vấn sinh mask + Hungarian matching	Cần instance/panoptic trong một mô hình	Số truy vấn cố định; ảnh nhiều instance hơn số truy vấn thì mất im lặng



Hình 1.12. Khung truy vấn của Mask2Former. Mô hình không hỏi từng pixel “bạn thuộc lớp nào” mà đưa ra một tập cố định truy vấn, mỗi truy vấn tự nhận một vùng và một nhãn; huấn luyện ghép các truy vấn với chân trị bằng ghép cặp Hungary như DETR. Vì đơn vị được nói tới là *một mask kèm một nhãn*, cả ba bài toán phân vùng chỉ còn khác nhau ở bước đọc tập mask đó, nên chúng dùng chung một mô hình và một hàm mất mát.

Hình 1.12 làm rõ điều mà hộp trực giác ở đầu mục chỉ nói bằng lời: cái được hợp nhất không phải ba kiến trúc mà là ba *cách phát biểu đầu ra*.

B. Một phép tính về cái giá của độ phân giải (The Resolution Budget)

Bảng 1.2 nói “bộ nhớ tăng mạnh” — con số cụ thể đáng ghi nhớ. Với ảnh 1024×1024 và một tầng đặc trưng 256 kênh: ở output stride 16, feature map là $64 \times 64 \times 256 \approx 1,05$ triệu phần tử, tức 4,2 MB ở fp32. Hạ xuống output stride 8, kích thước không gian nhân đôi ở cả hai chiều nên số phần tử nhân bốn: 16,8 MB cho *một* tensor. **Lượt ngược** (*backward pass*) phải giữ kích hoạt của mọi tầng, nên hệ số bốn này nhân vào toàn bộ phần sâu của mạng. Đây là lý do rất thực tế khiến hệ thống triển khai hiếm khi chạy ở output stride 4. Nó cũng giải thích vì sao U-Net rẻ hơn nhiều so với cảm giác ban đầu: U-Net chỉ giữ đặc trưng phân giải cao ở các tầng *ngang*, nơi số kênh còn ít.

Ba tầng cùng giãn 2



một nửa số pixel không bao giờ được nhìn — hiệu ứng lưới

Ba tầng giãn 1, 2, 3



mọi pixel đều đóng góp, cùng trường tiếp nhận

(một chiều, kernel 3×1 , tâm ở giữa; ô tô đậm là pixel thực sự ảnh hưởng tới đầu ra ở tâm)

Hình 1.13. Hiệu ứng lưới của tích chập giãn, vẽ trên một hàng pixel. Khi mọi tầng dùng cùng tỉ lệ giãn chẵn, các vị trí lấy mẫu luôn rơi vào cùng một lớp chẵn lẻ, nên trường tiếp nhận trông thì rộng mà thực chất thừa và bỏ sót một nửa tín hiệu. Chọn dãy tỉ lệ nguyên tố cùng nhau làm các lớp dư khác nhau phủ lẫn nhau, và trường tiếp nhận trở lại đặc.

Hình 1.13 là ví dụ nhỏ nhất cho thấy vì sao “mở trường tiếp nhận” và “nhìn thấy mọi pixel” là hai chuyện khác nhau.

Về atrous: một kernel 3×3 với tỉ lệ giãn r có trường tiếp nhận hiệu dụng $3 + 2(r - 1)$. Xếp chồng ba tầng giãn 1, 2, 4 cho trường tiếp nhận $3 \rightarrow 7 \rightarrow 15$ mà không hề giảm độ phân giải. Nhưng nếu ba tầng cùng giãn 2, các vị trí lấy mẫu luôn rơi vào cùng một lớp chẵn lẻ. Một nửa số pixel khi đó không bao giờ được nhìn tới. Đó là hiệu ứng gridding, và cách chữa là chọn dãy tỉ lệ giãn nguyên tố cùng nhau.

C. Giới hạn và trường hợp hỏng

Chế độ hỏng nguy hiểm nhất của segmentation không nằm ở mô hình mà ở thước đo. **giao trên hợp trung bình** (*mean intersection over union, mIoU*) trung bình theo lớp, nên một lớp hiếm nhưng quan trọng (người đi bộ, vật cản nhỏ) đóng góp bằng một lớp phủ kín ảnh (bầu trời), điều này tốt; nhưng bên trong mỗi lớp, IoU trung bình theo diện tích, nên sai vài pixel ở biên của một vùng lớn gần như không bị phạt. Hệ quả là một mô hình có mIoU 0,85 vẫn có thể cho biên răng cưa tới mức không bám nổi mép đường. Nó cũng có thể bỏ sót hoàn toàn một vật cản 20×20 pixel mà chỉ mất một phần nghìn điểm. Nếu ứng dụng của bạn quan tâm tới biên hay tới vật nhỏ, mIoU là thước đo sai và phải bổ sung metric theo biên hoặc theo instance.

→ **hệ quả của** B/07 — một con số tổng luôn che giấu chế độ khó; cách chữa duy nhất là báo cáo theo lát, ở đây là lát theo kích thước vật và lát theo độ dài biên.

Ba chế độ hỏng còn lại đáng nêu tên riêng:

- **Vật mảnh.** Dây điện, chân bàn, cột — biến mất ở output stride lớn, và không có hậu xử lý nào lấy lại được thứ đã không được lấy mẫu. *Dấu hiệu:* IoU của lớp đó gần 0 trong khi mIoU tổng vẫn cao.
- **Nhiều instance cùng lớp chồng lấn.** Các mô hình theo truy vấn chia sẻ một ngân sách cố định, nên phần vượt ngân sách mất im lặng — đúng chế độ hỏng của DETR ở Mục 2, xuất hiện lại ở miền mask.
- **Khái niệm không ổn định ở mô hình nhận prompt.** Cùng một ảnh, hai prompt gần như đồng nghĩa cho ra hai phân vùng ở hai mức chi tiết khác nhau (cái cửa, hay tay nắm cửa?). Hệ thống hạ nguồn giả định “một vật, một mask” sẽ hỏng theo cách rất khó tái hiện, vì lỗi phụ thuộc câu chữ chứ không phụ thuộc ảnh.

CẦN NHỚ

Đừng nhớ danh sách kiến trúc; hãy nhớ rằng mọi kiến trúc segmentation đều là một câu trả lời cho cùng một câu hỏi “chi tiết không gian đã mất ở đâu, và lấy lại bằng cách nào”. Skip lấy lại từ tầng nông, atrous không cho mất, attention học lại quan hệ xa, query đổi hẳn đơn vị được nói tới. [T3]

D. Kết nối

[C/14-bai-toan-cv-loi/02-detection-mot-giai-doan](#) là tiền đề của phần Mask2Former: nếu chưa hiểu vì sao Hungarian matching bỏ được NMS thì khung truy vấn ở đây chỉ là một kiến trúc lạ. [C/13-kien-truc-backbone/03-vit-va-inductive-bias](#) giải thích cái giá dữ liệu mà SegFormer phải trả. [C/18-pretraining-va-foundation](#) là nơi bàn kỹ về bước dịch chuyển sang mô hình nền tảng và kinh tế học của nó. [C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc](#) nối tiếp trực tiếp: một mô hình segmentation per-frame có mIoU cao vẫn cho video nhấp nháy, và lý do là mô hình này chưa hề có khái niệm thời gian.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao nội suy ngược từ feature ở output stride 32 không thể tạo lại biên sắc, dù ta dùng phép nội suy tốt đến đâu?
2. U-Net và DeepLab tấn công cùng một vấn đề từ hai phía nào, và trong tình huống nào thì cách của U-Net tốt hơn hẳn?
3. Ba tầng atrous cùng tỉ lệ giãn 2 gây ra chuyện gì, và vì sao chọn tỉ lệ nguyên tố cùng nhau lại chữa được?
4. Vì sao việc semantic, instance và panoptic hợp nhất được vào một mô hình lại là bằng chứng cho thấy phát biểu bài toán đúng, chứ không chỉ là tiện lợi kỹ thuật?
5. Mô hình của bạn đạt mIoU 0,85 nhưng bỏ sót vật cản nhỏ trên đường. Vì sao metric không phát hiện được, và bạn thay bằng metric nào?

Nguồn

- [1] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.
- [2] R. Girshick, “Fast R-CNN,” in *ICCV*, 2015.
- [3] S. Ren, K. He, R. Girshick, et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *NeurIPS*, 2015.
- [4] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [5] K. He, G. Gkioxari, P. Dollár, et al., “Mask R-CNN,” in *ICCV*, 2017.
- [6] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [7] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [8] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [9] W. Liu, D. Anguelov, D. Erhan, et al., “SSD: Single Shot MultiBox Detector,” in *ECCV*, 2016.
- [10] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [11] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [12] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [13] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [14] Z. Ge, S. Liu, Z. Li, et al., “OTA: Optimal Transport Assignment for Object Detection,” in *CVPR*, 2021.
- [15] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [16] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [17] X. Zhu, W. Su, L. Lu, et al., “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *ICLR*, 2021.
- [18] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [19] H. Zhang, F. Li, S. Liu, et al., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection,” in *ICLR*, 2023.
- [20] J. Long, E. Shelhamer, and T. Darrell, “Fully Convolutional Networks for Semantic Segmentation,” in *CVPR*, 2015.
- [21] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [22] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.

- [23] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,” *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [24] E. Xie, W. Wang, Z. Yu, et al., “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers,” in *NeurIPS*, 2021.
- [25] B. Cheng, I. Misra, A. G. Schwing, et al., “Masked-Attention Mask Transformer for Universal Image Segmentation,” in *CVPR*, 2022.
- [26] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [27] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [28] K. Sun, B. Xiao, D. Liu, et al., “Deep High-Resolution Representation Learning for Human Pose Estimation,” in *CVPR*, 2019.
- [29] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [30] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [31] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [32] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.

1.4 Các bài toán lân cận, mỗi cái một đánh đổi riêng (Neighbouring Tasks)

PHỤ THUỘC C/14-bai-toan-cv-loi/01-detection-hai-giai-doan và 03-segmentation (RoIAlign, heatmap, độ phân giải); B/07-chia-du-lieu-va-danh-gia (slice, metric theo nhóm); B/08-dich-chuyen-phan-phoi (long-tail).

TÓM TẮT

Mục này gom bốn họ bài toán mà kỹ sư gặp hằng ngày nhưng không nằm gọn trong trục khung/pixel: ước lượng tư thế, OCR, học biểu diễn cho truy hồi, và hai chế độ phân bố dữ liệu (fine-grained, long-tail). Điểm chung của cả bốn là quyết định quan trọng nhất không phải chọn kiến trúc mà là *chọn dạng đầu ra*: điểm, heatmap, chuỗi, hay embedding. Dạng đầu ra kéo theo luôn cả loss, metric và chế độ hỏng. Đọc xong bạn phát biểu được một bài toán thì giác lạ dưới dạng đúng ngay từ đầu, thay vì phát hiện mình chọn sai sau khi đã huấn luyện xong. Nếu chỉ nhớ một điều: chọn sai dạng đầu ra thì không kiến trúc nào cứu được, và triệu chứng của nó luôn xuất hiện ở chỗ mô hình phải biểu diễn sự *mơ hồ* — khớp bị che, ký tự nhoè, hai người rất giống nhau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **dạng đầu ra** — kiểu dữ liệu mà mô hình phải sinh ra — một nhãn, một khung, một mask, một điểm, một chuỗi ký tự, hay một vector. Nó quyết định luôn hàm mất mát và thước đo, nên đây là quyết định thiết kế đắt giá nhất.
- **hồi quy (regression) và phân loại (classification)** — hồi quy là dự đoán một số thực (toạ độ x); phân loại là chọn một trong các lựa chọn rời rạc (ô lưới nào chứa khớp). Hai cách này hỏng theo kiểu rất khác nhau khi câu trả lời mơ hồ.
- **keypoint và pose** — keypoint là một điểm có ý nghĩa trên vật (khủy tay, góc biến số); pose là tập hợp các keypoint của một đối tượng cùng quan hệ giữa chúng.
- **heatmap** — một bản đồ xác suất cùng cỡ với feature map, giá trị cao ở nơi mô hình tin rằng có keypoint. Khác với một cặp toạ độ, heatmap biểu diễn được “có hai khả năng”.

- **embedding** — một vector số chiều cố định biểu diễn một ảnh, học sao cho hai ảnh cùng danh tính thì gần nhau. Nhận dạng trở thành đo khoảng cách, nên thêm một người mới không cần huấn luyện lại.
- **tập lớp mở (open-set)** — tình huống mà các lớp lúc chạy thật không có trong tập huấn luyện — lý do phân loại softmax không dùng được và metric learning ra đời.
- **long-tail** — phân bố dữ liệu mà vài lớp chiếm gần hết số mẫu còn hàng trăm lớp chỉ có vài chục mẫu; accuracy tổng bị các lớp đồng thống trị nên che kín phần đuôi.
- **lát (slice)** — một tập con dữ liệu chia theo một thuộc tính có ý nghĩa — theo lớp, theo kích thước vật, theo điều kiện chụp. Báo cáo metric theo lát là cách duy nhất thấy được các chế độ khó.

Đặt vấn đề

Ba mục trước đi theo hai trục quen thuộc: nhãn ở mức khung và nhãn ở mức pixel. Nhưng phần lớn hệ thống thị giác thật lại đứng ở chỗ khác. Cánh tay robot cần *toạ độ khớp* chứ không cần khung; hệ đọc biển số cần một *chuỗi ký tự* đi qua ba khối nối tiếp; hệ nhận dạng khuôn mặt cần trả lời “có phải cùng một người” cho những người chưa từng có trong tập huấn luyện, tức đầu ra không thể là một nhãn trong tập cố định. Mục này gom bốn họ bài toán như vậy.

↪ **tiền đề** B/05 — chọn dạng đầu ra chính là bước thứ hai trong bộ ba đầu vào – đầu ra – tín hiệu giám sát; sai ở bước này thì mọi lựa chọn sau đó đều sai theo.

Chúng không nổi thành một mạch lịch sử duy nhất, nhưng chúng chia sẻ một điểm chung đáng học: ở mỗi họ, quyết định quan trọng nhất là *chọn dạng đầu ra*, và chọn sai dạng đầu ra thì không kiến trúc nào cứu được.

Học xong mục này bạn làm được gì mà trước đó không làm được? Nhận một yêu cầu mơ hồ kiểu “đọc số công-tơ từ camera” hay “nhận ra người quen ở cửa”, bạn viết ngay được bộ ba đầu vào – đầu ra – tín hiệu giám sát cùng metric đúng. Kèm theo là danh sách chỗ hệ thống sẽ hỏng.

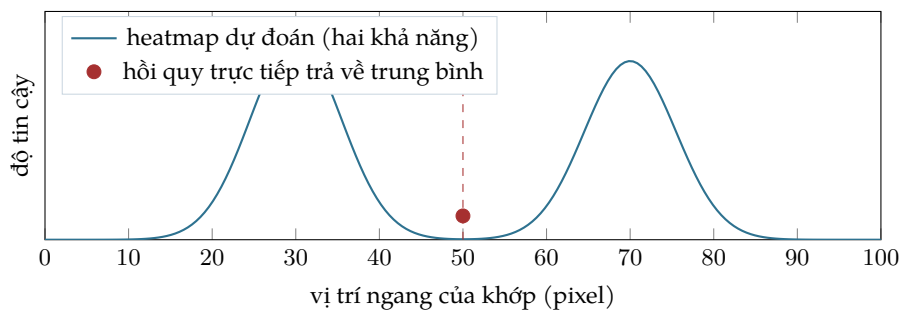
TRỰC GIÁC

Phân loại xây một cánh cửa cho mỗi lớp: muốn thêm lớp thì phải xây thêm cửa, tức huấn luyện lại. Metric learning không xây cửa mà vẽ *bản đồ*: mọi ảnh được đặt vào một không gian sao cho khoảng cách phản ánh “cùng danh tính hay không”. Thêm một người mới chỉ là ghim thêm một điểm móc lên bản đồ đã có. Sức mạnh và điểm yếu đều nằm ở đó: bạn không phải xây lại gì cả, nhưng nếu tẩm bản đồ vẽ sai thì mọi điểm móc ghim lên nó đều sai theo, và không có cách nào sửa cục bộ.

A. Keypoint và pose: hai lần chọn dạng (Keypoint and Pose Estimation)

Ước lượng tư thế phải chọn hai lần. Lần thứ nhất là *dạng đầu ra của một khớp*, và chỉ có hai phương án.

- **Hồi quy trực tiếp hai số (x, y)** — hiển nhiên và rẻ nhất. Nhưng mạng phải nén một bản đồ tin cậy hai chiều thành một điểm, qua các tầng fully connected. Cách này không biểu diễn được sự mơ hồ: khi một khớp bị che, câu trả lời đúng là “có hai khả năng”, còn hồi quy trả về trung bình của hai khả năng — một điểm mà cả hai giả thuyết đều bác bỏ.
- **Dự đoán một bản đồ nhiệt (heatmap)** — một bản đồ xác suất cho mỗi khớp, lấy argmax làm vị trí [28]. Heatmap giữ được sự mơ hồ. Huấn luyện cũng ổn định hơn nhiều vì bài toán trở thành phân loại theo pixel. Cái giá là bộ nhớ, và một lỗi lượng tử hoá đi kèm.



Hình 1.14. Vì sao dạng đầu ra quyết định chế độ hồng. Khi một khớp bị che, câu trả lời đúng là “một trong hai chỗ này”. Heatmap biểu diễn được điều đó bằng hai đỉnh, và tầng sau có thể chọn hoặc hoãn quyết định. Hồi quy trực tiếp bị hàm mất mát bình phương kéo về trung bình của hai khả năng — một điểm mà cả hai giả thuyết đều bác bỏ. Đây đúng là lý do một bộ ước lượng trạng thái không bao giờ thay một phân bố đa đỉnh bằng kỳ vọng của nó.

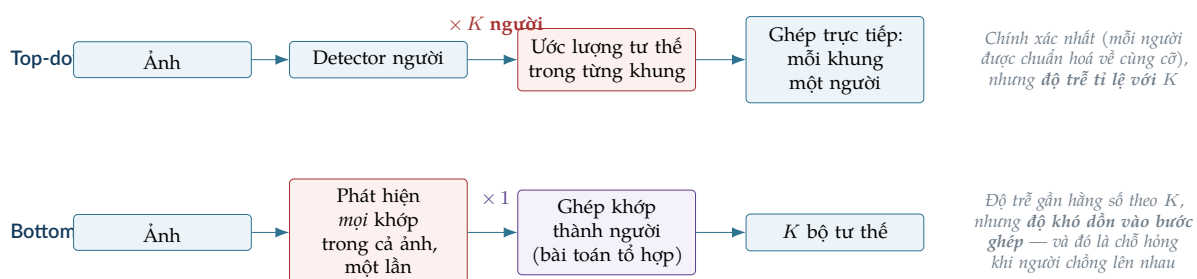
Hình 1.14 là ví dụ nhỏ nhất cho luận điểm trung tâm của mục này.

Lỗi lượng tử hoá đó tính được bằng tay. Với heatmap ở stride 4 trên ảnh 256×192 , lưới đầu ra là 64×48 ; argmax chỉ định vị được tới bội số của 4 pixel, nên sai số hệ thống là ± 2 pixel ngay cả khi mô hình hoàn hảo. Với người cao 200 pixel trong ảnh, 2 pixel là 1% — chấp nhận được; nhưng với bàn tay 40 pixel thì đó là 5%, và mọi bản triển khai nghiêm túc đều phải thêm một nhánh hồi quy độ lệch dưới pixel. Về bộ nhớ: $17 \text{ khớp} \times 64 \times 48 \times 4 \text{ byte} \approx 209 \text{ KB}$ cho một người — không đáng kể; nhưng ở độ phân giải 384×288 và 133 điểm (cả bàn tay và mặt) thì con số nhân lên hơn hai chục lần, và đó là lúc lựa chọn bắt đầu đau.

Lần chọn thứ hai là *cách xử lý nhiều người*. **top-down** chạy detector trước rồi ước lượng tư thế trong từng khung. Cách này chính xác nhất, vì mỗi người được chuẩn hoá về cùng kích thước. Nhưng chi phí tỉ lệ tuyến tính với số người, nên độ trễ phụ thuộc nội dung cảnh — một tính chất tệ hại cho hệ thời gian thực. **bottom-up** phát hiện mọi khớp trong cả ảnh một lần rồi ghép chúng thành người, chẳng hạn bằng trường liên kết từng phần trong OpenPose [29]: thời gian gần như hằng số theo số người, nhưng bước ghép là một bài toán tổ hợp và nó chính là chỗ hồng khi người chồng lên nhau.

↪ cùng nguyên lý với C/15/03 — top-down và bottom-up lặp lại đúng dạng của tracking-by-detection và joint tracking: tách rời thì dễ gỡ lỗi nhưng chi phí tỉ lệ số đối tượng, gộp chung thì chi phí ổn định nhưng độ khó dồn vào một bước liên kết mờ ám.

Đây là cùng một cấu trúc đánh đổi mà bạn sẽ gặp lại ở tracking: tách rời thì dễ gỡ lỗi nhưng chi phí tỉ lệ với số đối tượng; gộp chung thì ổn định về chi phí nhưng đẩy độ khó vào một bước liên kết mờ ám.



Hình 1.15. Cùng một bài toán, hai chỗ đặt độ khó. Top-down mua độ chính xác bằng cách chạy lại khối nặng cho từng người, nên chi phí — và quan trọng hơn, *phương sai* của chi phí — phụ thuộc nội dung cảnh. Bottom-up cố định chi phí phát hiện nhưng phải giải một bài toán ghép tổ hợp mà không có lời giải chắc chắn. Đây đúng là cấu trúc đánh đổi sẽ trở lại ở tracking dưới tên tracking-by-detection và joint tracking.

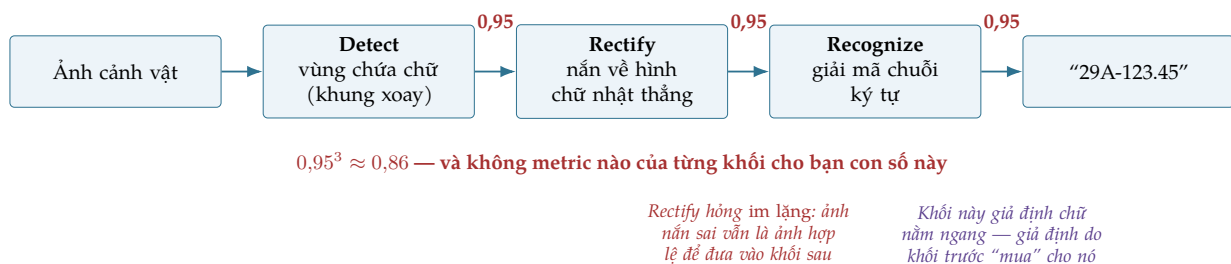
Hình 1.15 cho thấy vì sao lựa chọn này là lựa chọn hệ thống chứ không phải lựa chọn độ chính xác: với ngân sách mili-giây cố định, cột phải mới là cột quyết định.

B. OCR: một pipeline hoàn chỉnh trong một hệ thống nhỏ (Optical Character Recognition)

OCR đáng được mổ xẻ không phải vì nó khó nhất mà vì nó là hệ thống nhỏ nhất chứa đủ mọi thứ: phát hiện, biến đổi hình học, mô hình chuỗi, và một thước đo cấp hệ thống. Pipeline điển hình gồm ba khối nối tiếp:

1. **Detect** vùng chứa chữ. Thường là segmentation hoặc detection với khung xoay, vì chữ hiếm khi nằm ngang.
2. **Rectify** nắn vùng đó về một hình chữ nhật thẳng, bằng phép biến đổi phối cảnh hoặc một lưới biến dạng học được.
3. **Recognize** giải mã ảnh đã nắn thành chuỗi ký tự.

Điều đáng học nằm ở chỗ nối. Thứ nhất, khối rectify tồn tại vì khối recognize giả định chữ nằm ngang và đọc từ trái sang phải; đó là một giả định được đưa vào để làm khối cuối rẻ đi, đúng theo nguyên lý “độ khó không biến mất, chỉ bị đẩy đi chỗ khác”. Thứ hai, lỗi của ba khối nhân lên chứ không cộng lại: nếu mỗi khối đúng 95% thì hệ thống đúng $0,95^3 \approx 86\%$, và bạn không bao giờ đọc được con số đó từ metric của từng khối. Thứ ba, thước đo đúng của OCR là tỉ lệ khớp cả chuỗi hoặc khoảng cách chỉnh sửa, chứ không phải độ chính xác từng ký tự — một biến số sai một ký tự là một biến số sai. Bất kỳ ai từng dựng một pipeline thì giác nhiều tầng đều nhận ra hai bài học này, và OCR là nơi rẻ nhất để học chúng.

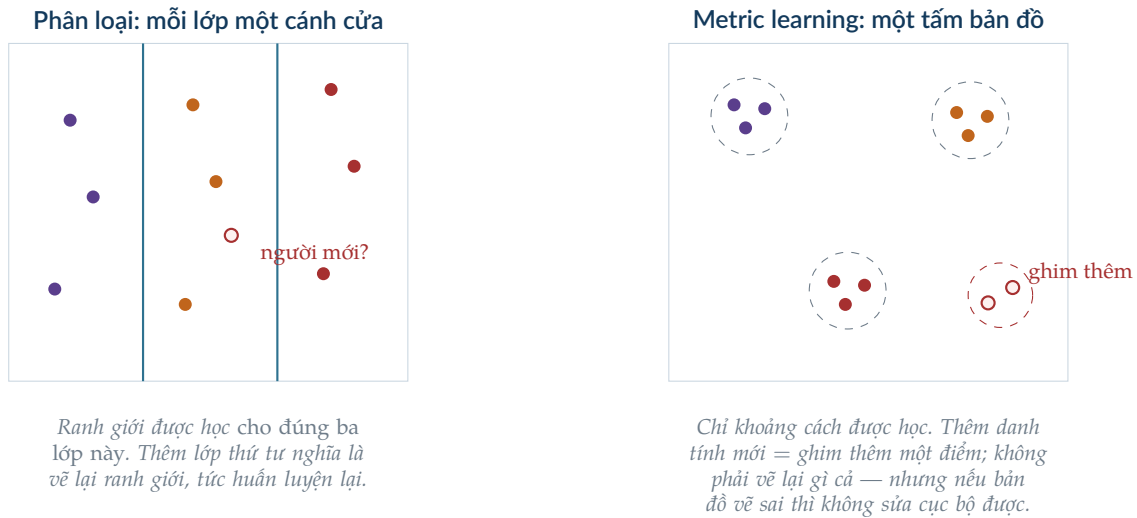


Hình 1.16. OCR là hệ thống nhỏ nhất chứa đủ mọi bệnh của một pipeline nhiều tầng. Độ chính xác của các khối *nhân* với nhau chứ không cộng, nên ba khối rất tốt vẫn cho một hệ thống tầm thường; khối giữa tồn tại chỉ để mua một giả định cho khối cuối; và thước đo đúng là khớp cả chuỗi chứ không phải độ chính xác từng ký tự — một biến số sai một ký tự là một biến số sai.

Hình 1.16 đáng nhớ vì nó áp dụng cho mọi pipeline nhiều tầng, kể cả pipeline SLAM: nếu bạn chỉ theo dõi metric của từng khối, bạn không bao giờ nhìn thấy *con số* mà người dùng thật sự trải nghiệm.

C. Học biểu diễn cho truy hồi (Metric Learning and Retrieval)

Nhận dạng khuôn mặt, re-identification và tìm kiếm ảnh có chung một ràng buộc mà phân loại không đáp ứng được: tập lớp *mở*. Không thể huấn luyện một softmax trên tập nhân viên của công ty rồi mỗi lần có người mới lại huấn luyện lại. Giải pháp là học một hàm *embedding* f sao cho khoảng cách trong không gian đích phản ánh quan hệ đồng nhất, rồi việc nhận dạng trở thành tìm láng giềng gần nhất. FaceNet huấn luyện bằng *triplet loss* — kéo cặp cùng danh tính lại gần hơn cặp khác danh tính ít nhất một biên α [30]; ArcFace thay bằng một biên góc cộng vào logit của softmax, vừa để huấn luyện hơn vừa cho ranh giới hình học rõ ràng trên mặt cầu đơn vị [31].



Hình 1.17. Vì sao tập lớp mở buộc phải đổi dạng đầu ra. Bên trái, mô hình học *ranh giới*, nên số lớp là một phần của kiến trúc và mọi lớp mới đòi một lần huấn luyện lại. Bên phải, mô hình chỉ học một phép đo khoảng cách, nên danh tính trở thành dữ liệu chứ không còn là tham số. Cái giá là chất lượng phụ thuộc hoàn toàn vào hình dạng của không gian nhúng — thứ không sửa được cho riêng một truy vấn.

Hình 1.17 giải thích luôn vì sao tăng tìm láng giềng xấp xỉ lại quan trọng đến thế: khi danh tính là dữ liệu, việc tra cứu trở thành một phần của hệ thống chứ không còn là một phép nhân ma trận.

Cái giá của cách tiếp cận này gồm ba phần. Thứ nhất, chất lượng phụ thuộc *hoàn toàn* vào không gian nhúng; không có cơ chế nào sửa được một truy vấn cụ thể mà không động vào toàn cục. Thứ hai, việc lấy mẫu quan trọng ngang hàm loss: phần lớn cặp ngẫu nhiên đã thỏa biên nên không sinh gradient, và mọi phương pháp thực dụng đều phải khai thác mẫu khó — lại là một chi tiết nằm trong code. Thứ ba, ở quy mô lớn phải thêm một tầng *approximate nearest neighbour* (ANN), và tầng đó có sai số riêng: recall của ANN đặt trần lên recall của cả hệ thống, độc lập với chất lượng embedding.

CẠM BẦY

Một khảo sát có kiểm soát đã chỉ ra rằng phần lớn “tiền bộ” công bố trong metric learning biến mất khi so sánh công bằng [32]. Công bằng ở đây nghĩa là cùng backbone, cùng cách lấy mẫu, cùng quy trình chọn siêu tham số. Bài học không chỉ dành cho metric learning: khi một lĩnh vực có nhiều bậc tự do nằm ngoài phần được viết trong bài (cách lấy mẫu, augmentation, lịch huấn luyện), bảng kết quả đo lường công sức tinh chỉnh nhiều hơn đo lường ý tưởng.

D. Fine-grained và long-tail: hai chế độ mà accuracy tổng che giấu

Hai chế độ khó cuối cùng không phải là bài toán mới mà là hai kiểu phân bố dữ liệu khiến mọi con số tổng trở nên vô nghĩa. **phân loại chi tiết** (*fine-grained classification*) là khi khác biệt giữa các lớp nhỏ hơn khác biệt bên trong một lớp: hai loài chim khác nhau ở màu một mảng lông, trong khi cùng một loài trông rất khác giữa tư thế bay và tư thế đậu. Hệ quả kỹ thuật là mô hình cần độ phân giải cao ở đúng vùng phân biệt và cần bất biến mạnh với tư thế — hai yêu cầu kéo ngược nhau. **phân bố đuôi dài** (*long-tail distribution*) là khi vài lớp chiếm phần lớn dữ liệu còn hàng trăm lớp chỉ có vài chục mẫu; accuracy tổng bị các lớp đầu thống trị nên có thể rất cao trong khi mô hình gần như không bao giờ dự đoán các lớp đuôi.

↔ cùng nguyên lý với B/08 — long-tail ở đây và dịch chuyển phân phối ở chương 8 có chung một cơ chế: phân bố lúc đánh giá không giống phân bố mà hệ thống thật sự gặp.

Điểm chung của hai chế độ là chúng *vô hình* với một con số tổng. Cách phát hiện duy nhất là chia dữ liệu theo lát và báo cáo metric trên từng lát — theo lớp, theo kích thước vật, theo điều kiện chụp. Đây chính là kỹ thuật ở B/07-chia-du-lieu-va-danh-gia, và lý do nó được nhắc lại ở đây là vì detection và segmentation có thêm một chiều lát riêng: kích thước vật. Bộ công cụ như [dbolya/tide](#) tách mAP thành lỗi phân loại, lỗi định vị, lỗi trùng lặp và lỗi nền [6], [7]; đọc bảng đó thường có ích hơn đọc thêm mười paper kiến trúc.

Bảng 1.3. Bốn họ bài toán lân cận, đọc theo trục “dạng đầu ra quyết định mọi thứ còn lại”.

Bài toán	Dạng đầu ra	Metric đúng	Hỏng khi nào
Pose	Heatmap hoặc toạ độ	PCK, OKS	Khớp bị che: hồi quy trả về trung bình của hai khả năng
OCR	Chuỗi ký tự	Khớp cả chuỗi, khoảng cách chỉnh sửa	Lỗi nhân qua ba khối; rectify hỏng im lặng
Retrieval	Embedding	Recall@ <i>k</i> , mAP truy hồi	Không gian nhúng sai thì không sửa cục bộ được; recall của ANN đặt trần
Fine-grained / long-tail	Nhãn (như phân loại)	Accuracy theo lớp và theo lát	Con số tổng bị các lớp đầu thống trị, che kín các lớp đuôi

Bảng 1.3 nên đọc từ cột thứ hai sang: chọn cột hai thì cột ba và cột bốn gần như bị quyết định theo, và đó chính là lý do bước chọn dạng đầu ra đáng dành thời gian hơn bước chọn backbone.

E. Giới hạn và trường hợp hỏng

Bốn họ trên chia nhau bốn chế độ hỏng, và điểm chung là cả bốn đều *im lặng*:

- **Pose — khớp bị che.** Heatmap trả một đỉnh yếu ở vị trí sai, và hệ thống hạ nguồn không phân biệt được “dự đoán yếu” với “không có”. *Cách chữa duy nhất*: truyền độ tin cậy đi cùng toạ độ, để tăng sau quyết định. Đây đúng là nguyên tắc mà bộ lọc trạng thái trong SLAM đã dùng từ lâu — không ai truyền một phép đo mà không truyền hiệp phương sai của nó.
- **OCR — nhân lỗi và thất bại im lặng.** Ba khối nhân xác suất đúng với nhau, và khối rectify hỏng vẫn sinh ra một ảnh hợp lệ để đưa vào khối nhận dạng, nên không có ngoại lệ nào được ném ra.
- **Retrieval — gallery trôi theo thời gian.** Chất lượng giảm dần mà không có tín hiệu nào lúc chạy, vì hệ thống không bao giờ biết câu trả lời đúng. *Cách phát hiện*: giữ một tập truy vấn cố định và đo lại định kỳ.
- **Fine-grained và long-tail — chính bạn.** Đo bằng một con số tổng rồi tin rằng hệ thống hoạt động. *Cách chữa*: báo cáo theo lát, không bao giờ chỉ báo cáo một số.

CẦN NHỚ

Ở mỗi bài toán lân cận, câu hỏi đầu tiên không phải “dùng kiến trúc nào” mà là “đầu ra nên có dạng gì” — điểm, heatmap, chuỗi, hay embedding. Dạng đầu ra quyết định loss, metric, chế độ hỏng và cả cách hệ thống hạ nguồn phải đối xử với sự mơ hồ. [T3]

F. Kết nối

B/05-thiet-ke-bai-toan là tiền đề khái niệm: chọn dạng đầu ra chính là bước hai trong bộ ba đầu vào – đầu ra – tín hiệu giám sát. B/07-chia-du-lieu-va-danh-gia và B/08-dich-chuyen-phan-phoi là chỗ xử lý nghiêm túc fine-grained và long-tail. C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc nối trực tiếp với phần top-down/bottom-up: tracking lặp lại đúng cấu trúc đánh đổi ở quy mô thời gian. C/18-pretraining-va-foundation giải thích vì sao không gian nhúng học từ pretraining quy mô lớn thường tốt hơn không gian học riêng cho một bài toán retrieval nhỏ.

▮ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hồi quy trực tiếp tọa độ khớp lại xử lý sai trường hợp khớp bị che, trong khi heatmap thì không?
2. Với heatmap stride 4, sai số lượng tử hoá là bao nhiêu pixel, và khi nào con số đó bắt đầu quan trọng?
3. Top-down chính xác hơn nhưng vì sao lại là lựa chọn tệ cho một hệ thống có ngân sách mili-giây cố định?
4. Ba khối OCR mỗi khối đúng 95%. Hệ thống đúng bao nhiêu, và vì sao metric từng khối không cho bạn biết điều đó?
5. Metric learning cho phép thêm lớp mới không cần huấn luyện lại. Cái giá của khả năng đó là gì, và nó lộ ra ở đâu khi hệ thống chạy thật?
6. Vì sao accuracy tổng che giấu cả fine-grained lẫn long-tail, và bạn thay nó bằng cách báo cáo nào?

Nguồn

- [1] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.
- [2] R. Girshick, “Fast R-CNN,” in *ICCV*, 2015.
- [3] S. Ren, K. He, R. Girshick, et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *NeurIPS*, 2015.
- [4] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [5] K. He, G. Gkioxari, P. Dollár, et al., “Mask R-CNN,” in *ICCV*, 2017.
- [6] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [7] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [8] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [9] W. Liu, D. Anguelov, D. Erhan, et al., “SSD: Single Shot MultiBox Detector,” in *ECCV*, 2016.
- [10] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [11] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [12] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [13] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [14] Z. Ge, S. Liu, Z. Li, et al., “OTA: Optimal Transport Assignment for Object Detection,” in *CVPR*, 2021.
- [15] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [16] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [17] X. Zhu, W. Su, L. Lu, et al., “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *ICLR*, 2021.
- [18] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [19] H. Zhang, F. Li, S. Liu, et al., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection,” in *ICLR*, 2023.
- [20] J. Long, E. Shelhamer, and T. Darrell, “Fully Convolutional Networks for Semantic Segmentation,” in *CVPR*, 2015.
- [21] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [22] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.

- [23] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,” *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [24] E. Xie, W. Wang, Z. Yu, et al., “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers,” in *NeurIPS*, 2021.
- [25] B. Cheng, I. Misra, A. G. Schwing, et al., “Masked-Attention Mask Transformer for Universal Image Segmentation,” in *CVPR*, 2022.
- [26] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [27] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [28] K. Sun, B. Xiao, D. Liu, et al., “Deep High-Resolution Representation Learning for Human Pose Estimation,” in *CVPR*, 2019.
- [29] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [30] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [31] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [32] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.

1.5 Thực hành

Chương này là chương dễ thực hành nhất của cả cây, vì mọi khẳng định trong đó đều kiểm chứng được trong vài giờ trên một GPU tầm trung. Hãy tận dụng: đọc mười paper detection không dạy bạn được bằng một lần tự tay đổi cách gán mẫu và nhìn con số nhúc nhích.

Repo và tài nguyên. Về detection, [roboflow/rf-detr](#) là dòng DETR thực dụng nhất tại thời điểm viết (giấy phép Apache 2.0, backbone DINOv2 nên khá bền với vật bị che); [ultralytics](#) YOLO11/YOLO26 nhanh và hợp với thiết bị biên nhưng dùng giấy phép **AGPL-3.0**, hãy kiểm tra kỹ trước khi đưa vào sản phẩm đóng; [lyuwenyu/RT-DETR](#) và RTMDet trong [open-mmlab/mmdetection](#) (MIT) là hai lựa chọn trung dung. Về segmentation, SAM 3 cho phân vùng theo prompt trên cả ảnh lẫn video, Mask2Former và SegFormer cho tập nhãn đóng, còn [MIC-DKFZ/nnUNet](#) là baseline y tế rất khó đánh bại. Open-vocabulary: Grounding DINO, Grounded-SAM. Khung làm việc: [facebookresearch/detectron2](#), [open-mmlab/msegmentation](#), [qubvel-org/segmentation_models.pytorch](#). Soi lỗi: [voxel51/fiftyone](#) để nhìn tận mắt, [dbolya/tide](#) để tách mAP thành các loại lỗi. Danh sách này chốt tại tháng 9/2026 và tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng (detectron2, mmdetection, fiftyone) bền hơn nhiều — hãy đầu tư thời gian theo tỉ lệ đó.

Câu hỏi kiểm tra hiểu. Vì sao DETR bản gốc hội tụ chậm hơn Faster R-CNN nhiều lần, và hai bản sửa lớn tấn công vào nguyên nhân nào? Nếu hai detector chênh nhau 2 mAP, bạn làm thế nào để biết chênh lệch đó đến từ kiến trúc hay đến từ cách gán mẫu và công thức huấn luyện? Khi nào một mô hình nền tảng zero-shot vẫn thua một mô hình nhỏ fine-tune, và dấu hiệu nào cho bạn biết điều đó trước khi đo?

Mini project (vài giờ đến hai ngày). Huấn luyện RF-DETR và một biến thể YOLO trên cùng một tập dữ liệu tự thu, với cùng ngân sách thời gian chứ không phải cùng số epoch. So sánh mAP, độ trễ trung bình và phương sai độ trễ, dung lượng mô hình, và giấy phép. Kết thúc

bằng một khuyến nghị viết cho một tình huống triển khai cụ thể, trong đó nêu rõ ràng buộc nào loại bỏ phương án nào.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Anchor	Khung chữ nhật mẫu, kích thước và tỉ số cạnh định sẵn, rải đều trên mọi vị trí của feature map. Mô hình đoán <i>độ lệch</i> so với anchor gần nhất chứ không đoán khung từ số không. Nó mã hoá thông kê hình dạng của tập dữ liệu gốc, nên đối miền ảnh là phải thiết kế lại.
●●● RPN	Mạng con trượt trên chính feature map của backbone, mỗi vị trí trả về độ khách quan và độ lệch khung. Nó thay selective search, và vì dùng chung backbone nên chi phí sinh đề xuất gần như bằng không.
●● Vì sao Fast R-CNN vừa nhanh hơn vừa chính xác hơn R-CNN?	Nhanh hơn vì backbone chạy một lần thay vì hai nghìn lần, đưa chi phí từ $O(N)$ lượt backbone về $O(1)$. Chính xác hơn vì phân loại và hồi quy khung gộp vào một hàm mất mát huấn luyện một pha, nên mọi khối cùng tối ưu cho mục tiêu cuối.
●● RoIAlign	Phép cắt đặc trưng theo vùng, lấy mẫu song tuyến tại toạ độ thực thay vì làm tròn về ô nguyên. Ở stride 16, làm tròn dịch biên tới 3 pixel ảnh, đủ để phá nhánh mask; bỏ nó đi thì nhánh mask mới chạy được.
●● Vì sao vật nhỏ bị thiệt kép trước khi có FPN?	Tầng sâu có ngữ nghĩa mạnh nhưng độ phân giải thô, tầng nông thì ngược lại. Vật nhỏ cần cả hai mà không tầng nào cho đủ. FPN gộp chúng bằng nhánh đi xuống cộng kết nối ngang 1×1 .
●● Recall đề xuất thấp mà độ chính xác trên đề xuất vẫn cao — nguyên nhân?	Tập anchor lệch miền: nó không phủ dài tỉ lệ hay tỉ số cạnh của dữ liệu mới. Phân biệt với detector kém bằng cách đo riêng recall của giai đoạn một; nếu recall đó thấp thì lỗi nằm ở anchor, không ở bộ phân loại.
●●● Triệt phi cực đại (NMS)	Bước hậu xử lý giữ khung điểm cao nhất rồi xoá mọi khung chồng lấn quá ngưỡng IoU. Nó không học được vì nó xét quan hệ <i>giữa</i> các dự đoán. Hổng trong cảnh đông: hai vật cùng lớp chồng nhau thì chắc chắn mất một.
●●● Gán mẫu	Quy tắc quyết định vị trí dự đoán nào phải học vật nào. Nó nằm trong định nghĩa hàm mất mát chứ không phải trong kiến trúc, nên đổi nó là đổi bài toán tối ưu. Đây là thành phần giải thích phần lớn chênh lệch giữa các detector.
●●● Focal loss	Cross-entropy nhân thêm hệ số $(1 - p_t)^\gamma$ để nén đóng góp của mẫu đã đúng. Với $\gamma = 2$, một anchor nền chắc chắn bị hạ hơn bốn bậc, nên 10^5 mẫu nền hết chi phối gradient. Hổng trên nhãn nhiễu: mẫu gán sai trông y hệt mẫu khó.
●● Vì sao chỉ detector một giai đoạn cần focal loss?	Vì giai đoạn đề xuất đã ngậm lọc hàng chục nghìn vị trí xuống còn vài trăm, tức đã cân bằng lại tập mẫu. Bỏ nó đi thì tỉ lệ nền trên vật quay về cỡ 10^3 tới 10^4 , và mẫu dễ nhấn chìm tín hiệu học.

●●● phải thuộc ●● nên nắm ● biết là có

Active recall — phần 2 (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
•• ATSS	Cách gán mẫu chọn ngưỡng IoU riêng cho từng vật: lấy k ứng viên gần tâm nhất, tính trung bình và độ lệch chuẩn IoU của chúng, dùng $\mu + \sigma$ làm ngưỡng. Nó cho thấy sau khi thống nhất gán mẫu, khác biệt anchor với anchor-free gần như biến mất.
•• SimOTA	Cách gán mẫu phát biểu việc chọn mẫu dương thành bài toán vận tải tối ưu trên toàn ảnh, chi phí là tổ hợp loss phân loại và loss hồi quy. Ưu điểm là phân xử được vùng chồng lấn; nhược điểm là vô nghĩa ở giai đoạn đầu huấn luyện nên phải kèm bước khởi động.
••• Ghép cặp Hungary	Phép gán một-một giữa N truy vấn và các vật thật, tìm phép ghép có tổng chi phí nhỏ nhất. Vì chỉ một truy vấn được thưởng cho mỗi vật, mô hình tự học không sinh trùng lặp, và NMS hết lý do tồn tại. Giá là hội tụ chậm.
• AP ₅₀ cao nhưng AP ₇₅ thấp — nguyên nhân?	Định vị kém, không phải phân loại kém. Thường vì NMS xếp hạng bằng điểm phân loại, nên khung tự tin về nhãn nhưng lệch vị trí xoá mất khung đúng hơn. Phân biệt bằng TIDE: nếu lỗi định vị chiếm phần lớn thì đúng là nguyên nhân này.
•• Output stride	Tỉ lệ thu nhỏ giữa ảnh và feature map. Output stride 32 nghĩa là mỗi ô đặc trưng đại diện cho một mảng 32×32 pixel, nên sai số định vị biên có sẵn cỡ chục pixel. Hạ nó một nửa thì lượng kích hoạt gấp bốn.
•• Vì sao tích chập giãn không làm mất độ phân giải?	Vì nó chèn khoảng trống giữa các trọng số thay vì thu nhỏ ảnh. Kernel 3×3 giãn r có trường tiếp nhận hiệu dụng $3 + 2(r - 1)$, trong khi số ô đầu ra giữ nguyên. Cái giá là bộ nhớ, vì độ phân giải không giảm.
•• Vì sao ba tầng cùng giãn 2 bỏ sót nửa số pixel?	Vì mọi vị trí lấy mẫu rơi vào cùng một lớp chẵn lẻ, nên trường tiếp nhận trông rộng mà thực chất thưa. Đó là hiệu ứng lưới, và cách chữa là chọn dãy tỉ lệ giãn nguyên tố cùng nhau.
••• Mask classification	Phát biểu coi đầu ra là một tập truy vấn, mỗi truy vấn trả một mask nhị phân cộng một nhãn lớp, ghép với chân trị bằng ghép cặp Hungary. Nhờ nó semantic, instance và panoptic chỉ còn khác nhau ở cách đọc tập mask, dùng chung một mô hình.
•• Heatmap	Bản đồ xác suất cho mỗi keypoint, lấy argmax làm vị trí. Nó biểu diễn được sự mơ hồ khi khớp bị che, điều mà hồi quy toạ độ không làm được. Giá là bộ nhớ, cộng sai số lượng tử hoá ± 2 pixel ở stride 4.
• Số tổng đẹp — mAP cao, mIoU 0,85 — nhưng lớp vật nhỏ gần như bằng 0?	Vì cả hai thước đo đều trung bình theo diện tích bên trong mỗi lớp, nên vật tí hon đóng góp gần như bằng không, và một lớp hiếm bị các lớp lớn che lấp. Phân biệt bằng cách báo cáo theo lát kích thước vật; nếu chỉ lát nhỏ nhất sập thì đúng là nguyên nhân này.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
<code>detectron2</code> , <code>mmdetection</code>	Hiện thực trực tiếp mạch hai giai đoạn: RPN, RoIAlign, FPN, và các bộ gán mẫu thay thế được	Hạ tầng, bền hơn tên mô hình dẫn đầu rất nhiều
Họ RT-DETR / RF-DETR (2024–2026)	Dùng dự đoán tập hợp và ghép cặp Hungary để bỏ hẳn NMS, cộng attention biến dạng để chữa hội tụ chậm	Dòng detector không NMS thực dụng tính tới 9/2026

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Ultralytics YOLO	Bộ gán mẫu động kiểu SimOTA/TAL là một trong những thứ đóng góp nhiều nhất vào chênh lệch giữa các phiên bản	Được cho là vậy dựa trên code; ghi chú phát hành không phải lúc nào cũng nêu rõ
Segment Anything (SAM, và SAM 3 tính tới 9/2026)	Segmentation theo prompt — bỏ ràng buộc tập nhãn cố định, giữ nguyên đầu ra dạng mask	Đang được dùng làm bộ gán nhãn bán tự động trong nhiều pipeline dữ liệu
nnUNet	U-Net với skip connection, cấu hình tự động theo tập dữ liệu; vẫn là baseline rất khó đánh bại trong ảnh y tế	Bằng chứng rằng kiến trúc 2015 chưa hề lỗi thời khi ràng buộc là ít dữ liệu
tide, fiftyone	Tách mAP thành lỗi phân loại, định vị, trùng lặp, nền — hiện thực của ý “một con số tổng che giấu chế độ hồng”	Công cụ chẩn đoán, không phải mô hình

MINI PROJECT SLAM · Lọc vật thể động bằng segmentation trước khi vào frontend SLAM · 1–2 ngày

Mục tiêu	trả lời một câu hỏi kỹ thuật cụ thể thay vì một câu hỏi học thuật: đưa một mô hình phân vùng có sẵn vào trước frontend để vứt bỏ đặc trưng nằm trên người đi lại, thì quỹ đạo có tốt lên <i>vượt sàn nhiều</i> của chính harness của bạn không?
Dữ liệu	một chuỗi EuRoC có người di chuyển trong khung hình, hoặc một chuỗi tự quay trong hành lang có người qua lại; chân trị lấy từ hệ bắt chuyển động hoặc từ một lần chạy tham chiếu. Mô hình phân vùng dùng sẵn, không huấn luyện lại — đây là bài toán tích hợp, không phải bài toán huấn luyện.
Các bước	(1) chạy mô hình phân vùng ngoại tuyến trên cả chuỗi, lưu mask nhị phân của các lớp động (người, xe) ra đĩa để không làm nhiều phép đo thời gian; (2) chèn một bước lọc vào ngay sau khối trích đặc trưng: bỏ mọi đặc trưng có tâm rơi vào mask; (3) chạy cấu hình gốc và cấu hình có lọc, <i>mỗi cấu hình ít nhất ba lần</i> , ghi ATE bằng <i>evo</i> ; (4) ghi thêm hai số phụ mỗi lần chạy: số đặc trưng còn lại trung bình mỗi khung hình, và số khung hình mất bám.
Xong khi	bạn có một bảng ATE trung bình và độ lệch chuẩn cho cả hai cấu hình trên ít nhất ba lần chạy mỗi bên, và phát biểu được một trong hai kết luận có bằng chứng: “cải thiện lớn hơn hai lần độ lệch chuẩn nên không phải nhiều”, hoặc “cải thiện nằm trong sàn nhiễu nên chưa kết luận được”. Kèm một câu giải thích dựa trên hai số phụ: nếu ATE tệ đi, có phải vì mask ăn mất quá nhiều đặc trưng ở vùng giàu kết cấu không?
Nối với	E/24 cho cách đọc chênh lệch trong nhiễu; B/07 cho giao thức đánh giá và việc không được chọn lần chạy đẹp nhất; kết quả của bài này là baseline cho mini project của chương 15.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì quy tắc gán mẫu nằm trong *định nghĩa* hàm mất mát chứ không trong mô hình: đổi nó là đổi bài toán tối ưu, trong khi đổi backbone chỉ đổi cách giải cùng một bài toán. ATSS cho thấy sau khi thống nhất gán mẫu, khác biệt anchor / anchor-free gần như biến mất.
- A2. Vì chạy backbone một lần cho phép gộp phân loại và hồi quy khung vào một hàm mất mát duy nhất huấn luyện một pha; ở R-CNN ba pha rời rạc, mỗi pha tối ưu cho mục tiêu của riêng nó nên không pha nào tối ưu cho kết quả cuối.
- A3. Vì NMS thao tác trên *quan hệ giữa các dự đoán*, trong khi hàm mất mát của các kiến trúc đó chấm điểm từng dự đoán độc lập — không có chỗ để gán gradient. Gỡ được nó đòi hỏi đổi phát biểu sang dự đoán tập hợp với ghép cặp một-một, chứ không phải thêm một khối mới.
- A4. Rằng chi tiết cài đặt có thể là biến quyết định chứ không phải chi tiết phụ, nên phần đáng soi kỹ nhất của một paper thường là code và bảng ablation, không phải sơ đồ kiến trúc ở trang đầu.

- A5. Rằng ta đã tìm được phát biểu đúng. Khi ba bài toán trở thành ba cách hậu xử lý cùng một tập mask, đơn vị mà mô hình nói tới là một mask kèm một nhãn. Đơn vị đó khớp với cấu trúc thật của bài toán, không còn là quy ước của từng code base.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn làm rơi chùm chìa khoá trong một căn phòng và nhờ một người máy tìm hộ. Cách hiển nhiên là bảo nó “nhìn cả phòng rồi nói chìa nằm ở đâu”, nhưng người máy không làm được: có hôm một chùm chìa, có hôm ba chùm, có hôm không có chùm nào, mà nó thì chỉ biết trả lời những câu hỏi có hình dạng cố định.

Mẹo của cả chương này là chia phòng thành hàng trăm nghìn ô vuông nhỏ rồi hỏi mỗi ô đúng hai câu giống hệt nhau: “trong ô này có thứ gì không” và “nếu có thì mép của nó cách đây bao xa”. Bây giờ mọi câu hỏi đều cùng một dạng, nên người máy học được. Toàn bộ lịch sử của lĩnh vực là chuyện đưa dần những việc con người vẫn phải làm tay vào bên trong phần mà máy tự chỉnh được. Những việc đó là: khoanh trước vùng đáng nhìn, chọn cỡ ô, và dọn các câu trả lời trùng nhau. Cái giá có hai khoản. Thứ nhất, hàng chục ô cạnh nhau cùng nói “có”, nên phải có một bước dọn trùng, và bước dọn ấy chính là chỗ hai chùm chìa nằm sát nhau bị nhập làm một. Thứ hai, cái luật quyết định ô nào *được phép* nói “có” cho chùm chìa nào là do người đặt ra, nằm ngoài phần máy học — và đổi luật đó thường làm kết quả thay đổi nhiều hơn đổi cả cái máy.

Chỗ không rút gọn được: vì sao một luật chọn ô lại tốt hơn luật khác thì tới nay chủ yếu vẫn là kết quả đo được chứ chưa có lời giải thích gọn; ai nói chắc chắn về điều này là đang nói quá.

Một câu: Chia bài toán thành hàng trăm nghìn câu hỏi giống hệt nhau thì máy học được — nhưng luật chấm điểm cho những câu hỏi ấy mới là thứ quyết định kết quả.

Tổng kết chương

Ta đã đi từ R-CNN, nơi mạng chỉ làm mỗi việc phân loại một mẫu ảnh, tới các hệ thống mà gần như mọi quyết định đều nằm trong tầm với của gradient. Trục xuyên suốt là ranh giới giữa phần học được và phần cố định, và mỗi thế hệ đẩy ranh giới đó xuống thấp hơn. Hai thứ còn sót lại là NMS và cách gán mẫu. Chúng sót lại không ngẫu nhiên: một cái thao tác trên quan hệ *giữa* các dự đoán, cái kia nằm trong chính định nghĩa của loss. Cả hai chỉ gỡ được bằng cách đổi phát biểu bài toán. DETR và Mask2Former là hai lần đổi phát biểu như vậy, và điều đáng chú ý là chúng dùng chung một ý tưởng: dự đoán một tập hợp và ghép một-một.

Bài học thực dụng thì nằm ở chỗ khác và quan trọng hơn: phần quyết định kết quả thường không phải phân được viết trong bài. Cách gán mẫu, tỉ lệ mẫu dương/âm, lịch huấn luyện, ngưỡng NMS — những thứ này giải thích chênh lệch mAP nhiều hơn hình vẽ kiến trúc trên trang đầu. Mang thói quen đó sang mọi chương sau: trước khi tin một cải tiến, hỏi xem có gì khác đã thay đổi cùng lúc.

Còn bỏ ngỏ hai hướng. Thứ nhất, mọi thứ trong chương này chạy trên một ảnh tĩnh; thêm chiều thời gian sinh ra ba vấn đề hoàn toàn mới và đó là C/15-thi-giac-khong-thoi-gian. Thứ hai, khi tập nhãn không còn cố định, câu hỏi chuyển từ “kiến trúc nào” sang “pretraining trên nguồn giám sát nào”, và đó là C/18-pretraining-va-foundation.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Trục của chương là nuốt dần heuristic vào trong mạng. Selective search, RoIPooling và NMS bị nuốt ở ba thời điểm rất khác nhau — điều gì ở mỗi heuristic quyết định nó dễ hay khó nuốt?
2. Quy tắc gán mẫu xuất hiện ở cả detection hai giai đoạn, một giai đoạn và Hungarian matching của DETR. Ba cách gán đó khác nhau ở giả định nào về quan hệ giữa dự đoán và chân trị, và giả định nào hỏng khi vật thể chen chúc?

3. Mất cân bằng lớp trong detection và mất cân bằng lớp trong segmentation đều được chữa bằng cách đổi hàm mất mát, nhưng vì hai lý do khác nhau. Hai lý do đó là gì, và vì sao cùng một liều focal loss lại nguy hiểm ở một bên hơn bên kia?
4. Hệ của bạn chạy detector rồi mới cắt vùng cho head tư thế hoặc OCR. Kiểu hỏng nào của giai đoạn đầu bị thước đo của từng khối che giấu, và bạn đo cả chuỗi bằng cách nào?
5. Hộp, mặt nạ, heatmap khớp và nhúng metric learning là bốn cách phát biểu đầu ra. Với mỗi cách, chỗ nào trong đó là một phép lượng tử hoá hay làm tròn, và cái giá của nó lộ ra ở loại vật thể nào?
6. Vì sao việc ba bài segmentation gộp được vào một mô hình lại là bằng chứng về *phát biểu bài toán*, trong khi việc detection vẫn tách hai nhánh anchor và anchor-free thì không nói lên điều tương tự?
7. Bạn nhận một bảng so sánh detector với mọi dòng cùng backbone. Ba chi tiết nào không nằm trong bảng vẫn đủ sức đảo thứ tự, và bạn hỏi tác giả câu nào trước?

Nguồn

- [1] R. Girshick, J. Donahue, T. Darrell, et al., “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *CVPR*, 2014.
- [2] R. Girshick, “Fast R-CNN,” in *ICCV*, 2015.
- [3] S. Ren, K. He, R. Girshick, et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *NeurIPS*, 2015.
- [4] T.-Y. Lin, P. Dollár, R. Girshick, et al., “Feature Pyramid Networks for Object Detection,” in *CVPR*, 2017.
- [5] K. He, G. Gkioxari, P. Dollár, et al., “Mask R-CNN,” in *ICCV*, 2017.
- [6] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [7] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.
- [8] J. Redmon, S. Divvala, R. Girshick, et al., “You Only Look Once: Unified, Real-Time Object Detection,” in *CVPR*, 2016.
- [9] W. Liu, D. Anguelov, D. Erhan, et al., “SSD: Single Shot MultiBox Detector,” in *ECCV*, 2016.
- [10] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [11] Z. Tian, C. Shen, H. Chen, et al., “FCOS: Fully Convolutional One-Stage Object Detection,” in *ICCV*, 2019.
- [12] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [13] S. Zhang, C. Chi, Y. Yao, et al., “Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection,” in *CVPR*, 2020.
- [14] Z. Ge, S. Liu, Z. Li, et al., “OTA: Optimal Transport Assignment for Object Detection,” in *CVPR*, 2021.
- [15] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [16] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.
- [17] X. Zhu, W. Su, L. Lu, et al., “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” in *ICLR*, 2021.
- [18] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.
- [19] H. Zhang, F. Li, S. Liu, et al., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection,” in *ICLR*, 2023.
- [20] J. Long, E. Shelhamer, and T. Darrell, “Fully Convolutional Networks for Semantic Segmentation,” in *CVPR*, 2015.
- [21] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [22] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.

- [23] L.-C. Chen, G. Papandreou, I. Kokkinos, et al., “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs,” *IEEE TPAMI*, vol. 40, no. 4, pp. 834–848, 2018.
- [24] E. Xie, W. Wang, Z. Yu, et al., “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers,” in *NeurIPS*, 2021.
- [25] B. Cheng, I. Misra, A. G. Schwing, et al., “Masked-Attention Mask Transformer for Universal Image Segmentation,” in *CVPR*, 2022.
- [26] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [27] A. Kirillov, E. Mintun, N. Ravi, et al., “Segment Anything,” in *ICCV*, 2023.
- [28] K. Sun, B. Xiao, D. Liu, et al., “Deep High-Resolution Representation Learning for Human Pose Estimation,” in *CVPR*, 2019.
- [29] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [30] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [31] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [32] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.